HiresHere · technical guide

How HiresHere works, line by line

Every technology, every module and the reasoning behind it: from reading 537 company job feeds each morning to showing a visitor in Tunis which remote jobs they can really apply for.

Written for Aymen Jeddou · October 2026 · repository AymenJeddou/hireshere
Code excerpts are copied from the source when this PDF is built, so they match the code exactly.

Contents

1. How to read this guide, and a glossary
2. The idea and the big picture
3. The technologies, one by one
4. The repository, file by file
5. Configuration: countries, boards, secrets
6. The data model and the database
7. Fetching jobs: HTTP, concurrency, connectors
8. Cleaning text and dates
9. The company list and the logos
10. Classification: asking an AI the right question
11. Running the AI on free tiers
12. The verdict engine: rules, not AI
13. Export: from the database to the website's data
14. Email alerts, welcome email and Telegram
15. The website: Astro, pages and languages
16. The browser code: search, filters, saved jobs
17. Cloudflare: Pages, Functions, D1 and the edge
18. Automation: the daily GitHub workflow
19. Testing, linting and evaluation
20. Design decisions and trade-offs
21. Working on the project: common tasks
22. Check your understanding

Chapter 1

How to read this guide, and a glossary

This guide explains the whole HiresHere system in the order data flows through it: from a company's job feed to a card on the website and an email in someone's inbox. Each chapter introduces a part of the system, shows the real code, and explains why it is written that way.

You do not need to read it in one go. Chapters 2 to 4 give the map; the later chapters can be read on their own when you work on that part. Code excerpts are real: they are pulled from the repository when the PDF is built, so if you change the code and rebuild the guide, the excerpts follow.

> **Boxes you will see**
>
> - **Blue boxes** explain a concept or a piece of background knowledge.
> - **Coral boxes** point out an idea worth remembering: a design choice and why it was made.
> - **Orange boxes** warn about a trap or a limit.
> - **Green boxes** suggest something to try yourself.

Glossary

ATS (applicant tracking system)
The software a company uses to publish jobs and receive applications: Greenhouse, Lever, Ashby, SmartRecruiters, Workable, Recruitee, Teamtailor. Most expose a public JSON "job board" feed, which is where HiresHere reads jobs from.

Board
One company's job feed on one ATS, identified by the pair (ats, slug), for example `(greenhouse, doximity)`. HiresHere tracks 537 of them.

Connector
A small Python module that knows how to read one ATS's feed and turn it into HiresHere's own `Job` objects.

Extraction
The structured answer the AI gives for one posting: remote type, allowed countries and regions, time zone window, work authorization, seniority, an evidence quote, and so on.

Verdict
The answer to "can someone living in country X apply?": *eligible* ("You can apply"), *likely*, *unclear* ("Ask the employer") or *not_eligible* (hidden). Computed by plain Python rules from an extraction, for each of 243 countries.

Content hash
A short fingerprint of a posting's title, location and description. If it does not change, the posting does not need to be read by the AI again.

Export
The step that turns the database into the JSON files the website reads.

Shard

One of 256 small JSON files holding the full details of the jobs whose slug hash starts with the same two hex characters. A job page loads only its shard.

Pages Function

A small piece of JavaScript Cloudflare runs on its servers for a specific URL, for example to build a job page on request.

D1

Cloudflare's hosted SQLite database, used for subscribers, error reports and employer requests.

Free tier

The amount of a paid service you can use without paying. HiresHere runs entirely on free tiers, which shapes many decisions.

Rate limit / quota

How many requests a service accepts per minute (rate limit) or per day (quota). Exceeding it returns HTTP 429.

Idempotent

An operation that gives the same result if you run it twice. The daily pipeline is built so that re-running it never double-counts or double-sends.

Chapter 2

The idea and the big picture

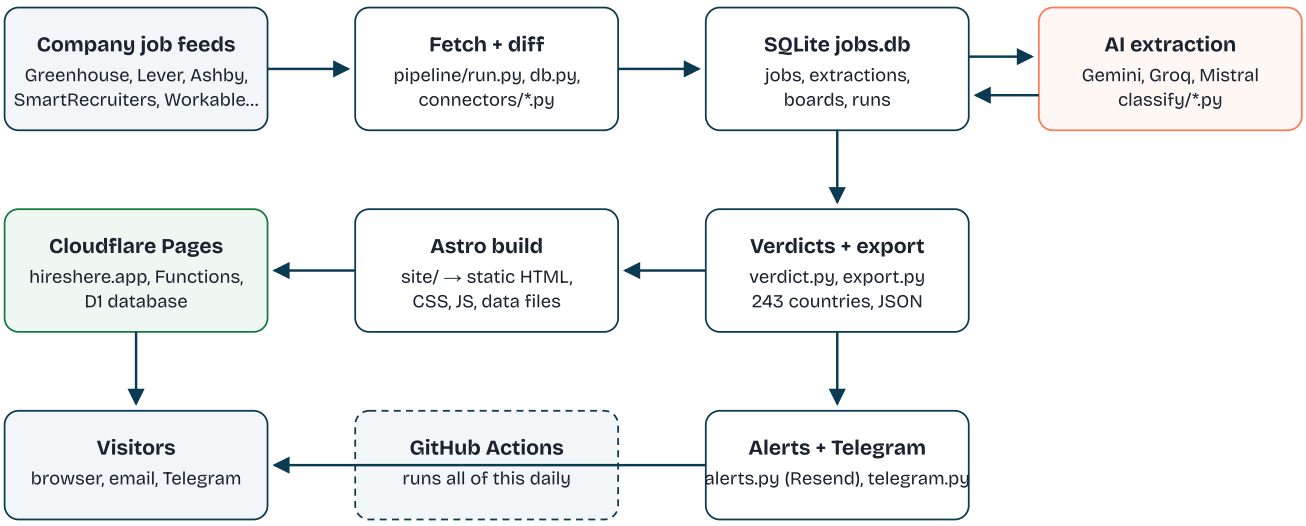
Most "remote" jobs are remote only inside one country or region. HiresHere reads each posting once, works out who may apply, and shows every visitor only the jobs open to where they live, with the sentence from the posting that proves it.

The problem in one example

A posting titled "Senior Data Engineer, Remote" may say, halfway down: "You must be based in the EU and authorized to work there." Someone in Tunisia reads the title, spends an evening on the application, and never hears back. Multiply that by hundreds of postings a week. HiresHere's job is to find that sentence for every posting and turn it into a clear answer per country.

The pipeline in six steps

1. **Fetch.** Read the public job feeds of 537 company boards on 7 ATS platforms, plus the Remote OK and Jobicy feeds. Store every open job in SQLite. Mark jobs that disappeared as expired.
2. **Pre-filter.** Skip jobs that are clearly office-only, so the AI is not spent on them.
3. **Extract.** Ask an AI model to read each remaining posting once and fill a strict schema: who may apply, from where, in which time zones, at what seniority, with an evidence quote.
4. **Decide.** Plain Python rules turn one extraction into a verdict for each of 243 countries. No AI here, so adding a country costs nothing.
5. **Export and publish.** Write compact JSON files and build a static website with Astro; deploy it to Cloudflare Pages.
6. **Notify.** Email subscribers a daily digest of new jobs open to their country, and post to Telegram channels.



Data flows left to right along the top, then back down to the visitor. GitHub Actions runs the whole chain once a day.

Three ideas that shape everything

1. Read each posting once, decide many times

The expensive, slow step is the AI reading a posting. So the AI produces a country-neutral *extraction* once per posting (stored by content hash), and cheap deterministic rules turn that into a verdict for any number of countries. When HiresHere went from 4 countries to 243, no posting had to be re-read.

2. Cautious by design

A wrong "You can apply" wastes someone's evening and destroys trust; an honest "Ask the employer" costs little. Every rule leans cautious, low-confidence AI answers are capped at "likely", and the evidence quote must really appear in the posting or the confidence drops.

3. Free tiers only

The whole system runs on free plans: GitHub Actions (free for public repositories), Cloudflare Pages, Functions and D1, Resend's free email plan, the free tiers of Gemini, Groq and Mistral. Many design choices (static pages, data shards, request pacing, one digest a day) exist to stay inside those limits.

What a visitor sees

- A search page with a country select in the header. Each job card shows one verdict pill for that country ("You can apply", "Likely open to you", "Ask the employer"), a short reason, and the date the employer posted it.
- A job page that opens with "You can apply from Brazil" (or the honest alternative), the reason, the exact quote, a world map coloured by verdict, and links to apply on the employer's site.
- Company pages, a getting-paid guide, an employer form, saved jobs and an application tracker kept in the browser, all in English, French and Arabic (right to left).
- Optional daily email alerts and per-country Telegram channels.

Chapter 3

The technologies, one by one

HiresHere deliberately uses few tools: the Python standard library plus two packages for the pipeline, Astro for the website, and free hosted services around them. This chapter explains what each one is and why it was chosen.

The data pipeline (Python)

Technology	What it is	What HiresHere uses it for
Python 3.12	The programming language of the pipeline.	Everything in `pipeline/`: fetching, database, AI calls, rules, export, emails.
uv	A fast Python package and project manager (installs dependencies, runs commands).	`uv sync` installs the exact versions in `uv.lock`; `uv run python -m pipeline.run fetch` runs a command inside that environment.
httpx	An HTTP client library with async support.	Every network call: job feeds, Gemini, Groq, Mistral, Resend, Telegram, Cloudflare's API.
Pydantic	Data validation with Python type hints.	The `Job` and `Board` models, and above all the `Extraction` schema that validates every AI answer.
asyncio (standard library)	Python's built-in way to run many network waits at the same time.	Fetching 537 boards five at a time; running two or three AI providers in parallel.
SQLite (standard library `sqlite3`)	A full SQL database stored in a single file.	`jobs.db`: every job ever seen, its status, the AI extractions, the run history.
argparse, csv, json, re, hmac, html.parser	Standard library modules.	The command line, the board list, JSON files, regular expressions for the pre-filter and rules, unsubscribe tokens, turning HTML into text.
pytest	The test runner.	75 tests, run offline against saved example feeds.
Ruff	A Python linter and formatter.	Keeps style consistent and catches mistakes: `ruff check` and `ruff format`.

> **Why so few dependencies?**
>
> The owner's rule is "stdlib plus httpx and pydantic". Fewer packages means fewer updates, fewer security issues, faster installs in CI, and code a newcomer can read without learning a framework. When something can be done in a few lines with the standard library (HTML to text, HMAC, CSV), it is.

The AI models

Provider	Model	Role
Google Gemini API	`gemini-3.1-flash-lite`	Main classifier. Supports a strict JSON schema for answers (`responseJsonSchema`) and a Batch API at half price. Used on the free tier since 2026-10-11.
Groq	`openai/gpt-oss-120b`	Free fallback that works the queue in parallel with Gemini. OpenAI-compatible API.

Mistral	`mistral-small-latest`	Third provider, wired in but idle until its free API plan is active on the account.

The website

Technology	What it is	Use in HiresHere
Astro	A static site generator: you write components, it outputs plain HTML at build time.	Every page under `site/src/pages`; 2,000+ static pages built in about 20 seconds.
TypeScript	JavaScript with types.	Browser scripts (search, filters, saved jobs, maps) and the shared rendering code for job pages.
CSS (no framework)	Hand-written styles with custom properties (design tokens).	`global.css` (tokens, base) and `components.css` (cards, pills, layouts).
Fontsource	Self-hosted web fonts as npm packages.	Bricolage Grotesque (headings) and DM Sans (text), served from the site itself, not from Google Fonts.
Natural Earth + `tools/world-map.mjs`	Public-domain map data and a small script that turns it into one SVG.	`public/world.svg`: one path per country, coloured in the browser by job counts or verdicts.

Hosting and services

Service	What HiresHere uses
Cloudflare Pages	Hosts the static site on Cloudflare's global network at hireshere.app; deploys come from `wrangler pages deploy`.
Pages Functions	Server code for job pages (rendered on request) and the `/api/*` endpoints (sign-up, unsubscribe, reports, employer form, visitor country).
Cloudflare D1	SQLite in the cloud: subscribers, error reports, employer leads.
Cloudflare extras	Email Routing (contact@hireshere.app to the owner's inbox), a Redirect Rule (www to the apex domain), Web Analytics (visitor counts and Core Web Vitals).
Wrangler	Cloudflare's command line tool: deploys, D1 queries, Pages secrets.
GitHub	The code repository, **Actions** (the daily workflow), **Releases** (the encrypted database snapshot), **Secrets** (API keys).
Resend	Sends email (daily digests and welcome emails) from alerts@hireshere.app; free plan: 100 emails a day.
Telegram Bot API	The @hireshere_bot posts new jobs to four country channels.
OpenSSL	Encrypts the database snapshot (AES-256) before it is stored on the public repository's release.

Chapter 4

The repository, file by file

The repository has two halves: `pipeline/` (Python, runs in GitHub Actions) and `site/` (Astro and Cloudflare Functions). They meet in a handful of JSON files that the pipeline writes and the site reads.

Repository layout (generated files and assets left out)

```
hireshere/
├── pipeline/                the data pipeline (Python)
│   ├── config.py            names, domain, limits, featured countries, WORLD
│   ├── models.py            Board, Job, FetchResult (Pydantic)
│   ├── http.py              polite HTTP client: User-Agent, timeout, retries
│   ├── db.py                SQLite schema and the five diff rules
│   ├── run.py               the command line: fetch, classify, export, alerts…
│   ├── text.py              HTML to text, dates from any feed format
│   ├── connectors/          one module per job source (9 of them)
│   ├── classify/
│   │   ├── schema.py        what the AI must extract (Extraction + JSON schema)
│   │   ├── prompt.py        the prompt, the pre-filter, the excerpt
│   │   ├── gemini.py        Gemini API: one call, batch submit/status/results
│   │   ├── compat.py        OpenAI-compatible free providers (Groq, Mistral)
│   │   ├── runner.py        triage, cost estimate, workers, storage, batch collect
│   │   ├── verdict.py       the rules: extraction + country → verdict + reason
│   │   └── evalset.py       hand-checked evaluation sample and scoring
│   ├── export.py            database → JSON for the website
│   ├── alerts.py            daily email digests (Resend)
│   ├── telegram.py          posts to the Telegram channels
│   ├── seed.py              builds and validates the company list
│   ├── logos.py             finds each company's website and icon
│   └── healthcheck.py       checks the live site after each deploy
├── data/
│   ├── boards.csv           the 537 company boards (ats, slug, name, website, source)
│   ├── countries.json       243 countries: names (en, fr, ar), regions, UTC offsets
│   ├── CREDITS.md           licences of the third-party lists
│   └── eval/                the evaluation sample
├── tests/                   pytest tests and saved example feeds (fixtures/)
├── site/                    the website
│   ├── astro.config.mjs     static build settings
│   ├── wrangler.jsonc       Cloudflare project and D1 binding
│   ├── schema.sql           D1 tables: subscribers, reports, employer_leads
│   ├── src/
│   │   ├── pages/           one file per URL (+ fr/ and ar/ versions)
│   │   ├── components/      Home, JobSearch, CompanyPage, Alerts…
│   │   ├── layouts/Layout.astro   header, footer, meta tags, language switch
│   │   ├── scripts/         browser TypeScript: cards, search, job page…
│   │   ├── styles/          global.css (tokens), components.css
│   │   ├── i18n.ts, i18n-ar.ts    all interface text in three languages
│   │   ├── data.ts          build-time access to the exported JSON
│   │   └── data/            meta.json, companies.json (written by export)
│   ├── public/              served as is: data/ (search index, shards), logos/,
│   │                        world.svg, _routes.json, _redirects
│   ├── functions/           Cloudflare Pages Functions (server code)
│   │   ├── _middleware.js   canonical host and old-URL redirects
│   │   ├── _job.js          renders a job page from its shard
│   │   ├── jobs/[slug].js   route for /jobs/<slug> (+ fr/, ar/)
│   │   └── api/             subscribe, unsubscribe, report, employer, geo
│   └── tools/world-map.mjs  generates world.svg from Natural Earth
├── docs/                    SPEC, ROADMAP, DECISIONS (the "why" log), this guide
├── .github/workflows/daily.yml    the daily automation
├── CLAUDE.md, DESIGN.md, PRODUCT.md   working notes, design system, product truth
└── pyproject.toml, uv.lock  Python project and exact dependency versions
```

The contract between the two halves

The pipeline and the website never import each other's code. They communicate through files written by `python -m pipeline.run export`:

File	Read by	Contents
`site/src/data/meta.json`	Astro at build time, Functions	Update time, product name, domain, counts, featured countries with verdict counts, all 243 countries with names and regions.
`site/src/data/companies.json`	Astro at build time	One entry per company: name, website, number of jobs, intern/junior count, countries named in its postings.
`site/public/data/search-<lang>.json`	The browser	One compact record per listed job (about 12,000), with verdict groups and reasons in that language.
`site/public/data/j/<xx>.json`	Job page Function, browser fallback	256 shards with each job's full details, same-company and similar jobs.

> **Why a file contract?**
> Because the two halves run in different places: the pipeline in GitHub Actions with Python, the site in the browser and on Cloudflare with JavaScript. Plain JSON files are the simplest thing both sides understand, they can be inspected with any text editor, and the site can be rebuilt from them without touching the database.

Documentation you should know about

- `docs/DECISIONS.md`: a dated log of every significant decision and its reason. When you wonder "why is it like this?", look there first.
- `docs/SPEC.md`: the original brief. `docs/ROADMAP.md`: what is done and what is open.
- `CLAUDE.md`: the owner's rules and the main commands. `DESIGN.md`: colours, type and components. `PRODUCT.md`: who the product is for.

Chapter 5

Configuration: countries, boards, secrets

Three kinds of configuration drive the pipeline: constants in `pipeline/config.py`, data files (`countries.json`, `boards.csv`), and secrets passed as environment variables. A rule of the project: countries and names are configuration, never `if country == "TN"` branches in the code.

pipeline/config.py

The single source of truth for the product name, the domain, how politely to fetch, and the AI settings. Nothing else hard-codes "HiresHere" or "hireshere.app": the site even receives the domain through the exported `meta.json`.

pipeline/config.py · lines 1 to 20

```
"""Single source of truth for names, endpoints etiquette and launch countries."""

import json
import os
from pathlib import Path

PRODUCT_NAME = "HiresHere"
DOMAIN = "hireshere.app"
CONTACT_EMAIL = f"contact@{DOMAIN}"  # Cloudflare Email Routing forwards it to the owner

USER_AGENT = f"{PRODUCT_NAME}-bot/0.1 (+https://{DOMAIN}; job eligibility index)"
MAX_CONCURRENCY = 5
RETRIES = 2
TIMEOUT_SECONDS = float(os.environ.get("HTTP_TIMEOUT", "20"))
MAX_CONSECUTIVE_FAILURES = 5
# Feeds that only list recent jobs: a missing job expires this many days after it was posted.
ROLLING_DAYS = {"jobicy": 30}

DB_PATH = os.environ.get("JOBS_DB", "jobs.db")
BOARDS_CSV = "data/boards.csv"
```

`USER_AGENT` identifies the bot honestly to every job feed, with a link back to the site. `MAX_CONCURRENCY = 5` means at most five feeds are downloaded at the same time, so no company's server is hammered. `ROLLING_DAYS` handles feeds that only list recent jobs (Jobicy): a job that disappears from such a feed may still be open, so it only expires 30 days after it was posted.

Featured countries and the world

`COUNTRIES` lists the four featured countries (Tunisia, Morocco, Algeria, Egypt) with names in three languages, one UTC offset and the region names that include them. `WORLD` merges every country from `data/countries.json` with those featured entries:

pipeline/config.py · WORLD

```
WORLD = {
    code: c | COUNTRIES.get(code, {})
    for code, c in json.loads(
        (Path(__file__).parent.parent / "data/countries.json").read_text("utf-8")
    ).items()
}
```

This is a **dict comprehension** with the merge operator `|`: for each country in the JSON file, start from its entry and overlay the featured entry if there is one. Featured countries therefore keep their single `utc_offset`, which kept their verdicts stable when the site went worldwide.

data/countries.json

243 entries keyed by ISO 3166-1 alpha-2 code. Each has names in English, French and Arabic, the regions it belongs to (used to match postings that say "EMEA" or "LATAM"), and the range of UTC offsets its time zones cover (from the IANA time zone database). Example:

data/countries.json · lines 2869 to 2882

```
"TN": {
"name": "Tunisia",
"name_ar": "تونس",
"name_fr": "Tunisie",
"regions": [
"Africa",
"EMEA",
"MENA",
"North Africa"
],
"tz": [
1.0,
1.0
]
```

The `tz` range matters for countries with several time zones: a posting that requires "US hours" (UTC-10 to UTC-5) fits Brazil (UTC-5 to UTC-2) because the ranges overlap.

The AI settings

pipeline/config.py · lines 64 to 75

```
    ).items()
}

# Phase 2 classifier. Prices are USD per million tokens for standard (non-batch) calls,
# from https://ai.google.dev/gemini-api/docs/pricing on 2026-10-08. Batch is half.
CLASSIFIER_MODEL = os.environ.get("CLASSIFIER_MODEL", "gemini-3.1-flash-lite")
# Requests per minute in sync mode, kept under the free tiers (0 = no pacing).
GEMINI_RPM = float(os.environ.get("GEMINI_RPM", "0"))
# Stop starting new requests after this many minutes in sync mode (0 = no limit).
CLASSIFY_MINUTES = float(os.environ.get("CLASSIFY_MINUTES", "0"))
PRICE_IN = float(os.environ.get("CLASSIFIER_PRICE_IN", "0.25"))
PRICE_OUT = float(os.environ.get("CLASSIFIER_PRICE_OUT", "1.50"))
```

Every value that might change between environments is read from an environment variable with a default (`os.environ.get("GEMINI_RPM", "0")`). The daily workflow sets the pacing; local runs use the defaults. `MAX_USD_WITHOUT_CONFIRM` is a safety net from the paid era: a batch estimated above $5 does not start without `--yes`.

data/boards.csv: the company list

data/boards.csv · lines 1 to 6

```
ats,slug,company_name,website,source_of_entry
ashby,1password,1Password,https://www.1password.com,remoteintech+ats-scrapers
```

```
ashby,3commas,3Commas,https://3commas.io/,ats-scrapers
ashby,airbyte,Airbyte,https://airbyte.com/,remoteintech+ats-scrapers
ashby,airgarage,AirGarage,https://www.airgarage.com/,remoteintech+ats-scrapers
ashby,airspace-intelligence.com,Air Space Intelligence,https://www.airspace-intelligence.com/,ats-scrapers
```

One row per company board: which ATS, the board's slug on that ATS (as in `boards.greenhouse.io/doximity`), the company's display name, its website (used for logos) and where the entry came from. The list is built and validated by `pipeline/seed.py` (chapter 9). Removing a row retires the board: its open jobs are expired on the next run.

Secrets and environment variables

Secrets never go into git. Locally they live in `.env` (ignored by git); in GitHub Actions they are repository secrets; for the Pages Functions they are Cloudflare Pages secrets. `.env.example` documents every variable:

.env.example

```
# Copy to .env for local runs. Never commit .env.

# Path of the SQLite database (default jobs.db)
JOBS_DB=jobs.db
# Per-request network timeout in seconds (default 20)
HTTP_TIMEOUT=20

# Phase 2 classifier (Gemini). Also set GEMINI_API_KEY as a GitHub Actions secret.
GEMINI_API_KEY=
# Default gemini-3.1-flash-lite. Prices in USD per million tokens, standard tier.
# CLASSIFIER_MODEL=gemini-3.1-flash-lite
# CLASSIFIER_PRICE_IN=0.25
# CLASSIFIER_PRICE_OUT=1.50

# Telegram channels (optional): bot from @BotFather, added as admin of each channel
TELEGRAM_BOT_TOKEN=
TELEGRAM_CHANNELS=TN=@channel_tn:fr,MA=@channel_ma:fr,DZ=@channel_dz:fr,EG=@channel_eg:ar

# Free fallback classifier (Groq, OpenAI-compatible), used once Gemini's free daily quota is out
GROQ_API_KEY=
# GROQ_MODEL=openai/gpt-oss-120b
# Pacing for free tiers in sync mode, requests per minute (0 = none)
# GEMINI_RPM=10
# GROQ_RPM=4

# Mistral free Experiment tier (choose the plan and verify a phone in console.mistral.ai)
MISTRAL_API_KEY=
# MISTRAL_MODEL=mistral-small-latest
# MISTRAL_RPM=50
```

For local runs, `pipeline/run.py` loads `.env` with a ten-line parser instead of a library. Real environment variables win, so CI settings are never overridden by a stray local file:

pipeline/run.py · load_dotenv

```
def load_dotenv(path: str = ".env") -> None:
    """Minimal .env support for local runs. Real environment variables win."""
    if os.path.exists(path):
        for line in open(path, encoding="utf-8"):
            key, sep, value = line.strip().partition("=")
            if sep and not key.startswith("#"):
                os.environ.setdefault(key.strip(), value.strip())
```

Secret	Where	Used by
GEMINI_API_KEY, GROQ_API_KEY, MISTRAL_API_KEY	GitHub, .env	The classifier.
CLOUDFLARE_API_TOKEN, CLOUDFLARE_ACCOUNT_ID	GitHub, .env	Deploying the site; D1 queries from the pipeline.
RESEND_API_KEY	GitHub, .env, Pages	Digests (pipeline) and welcome emails (sign-up Function).
UNSUBSCRIBE_SECRET	GitHub, Pages	Signing unsubscribe links (pipeline) and checking them (Function).
TELEGRAM_BOT_TOKEN, TELEGRAM_CHANNELS	GitHub	Posting to channels.
DB_PASSPHRASE	GitHub, .env	Encrypting and decrypting the database snapshot.

Chapter 6

The data model and the database

Everything the pipeline knows lives in one SQLite file, `jobs.db`. This chapter covers the Python models, the tables, and the five rules that keep the database in step with the job feeds.

Pydantic models (pipeline/models.py)

A **Pydantic model** is a class whose fields have types. Creating one validates the data: a missing required field or a wrong type raises an error instead of silently storing garbage. Connectors produce `Job` objects; the fetcher wraps each board's outcome in a `FetchResult`.

pipeline/models.py · Job

```
class Job(BaseModel):
    source: str
    board_slug: str
    source_job_id: str
    company_name: str
    title: str
    location_raw: str = ""
    remote_flag_raw: str | None = None
    department: str | None = None
    employment_type_raw: str | None = None
    salary_min: float | None = None
    salary_max: float | None = None
    salary_currency: str | None = None
    description_text: str = ""
    apply_url: str
    posted_at: str | None = None  # when the employer published it, from the feed

    @computed_field
    @property
    def id(self) -> str:
        return f"{self.source}:{self.board_slug}:{self.source_job_id}"

    @computed_field
    @property
    def content_hash(self) -> str:
        text = "\x1f".join([self.title, self.location_raw, self.description_text])
        return hashlib.sha256(text.encode()).hexdigest()[:16]
```

Two **computed fields** are derived, never stored by hand:

- `id` joins source, board and the ATS's own job id: `greenhouse:doximity:8132424`. It is stable across runs, so the same job is recognised every day.
- `content_hash` is the first 16 hex characters of a SHA-256 over title, location and description. If the employer edits the posting, the hash changes and the job goes back to the AI; if not, its stored extraction is reused.

> **What is a hash?**
>
> A hash function turns any text into a short fixed-length fingerprint. The same input always gives the same fingerprint, and a tiny change gives a completely different one. That makes it a cheap way to ask "did this posting change?" without comparing whole descriptions. The `\x1f` separator (an invisible "unit separator" character) keeps "ab" + "c" from hashing like "a" + "bc".

The tables (pipeline/db.py)

pipeline/db.py · SCHEMA

```
SCHEMA = """
CREATE TABLE IF NOT EXISTS boards (
    ats TEXT NOT NULL,
    slug TEXT NOT NULL,
    company_name TEXT NOT NULL,
    website TEXT,
    source_of_entry TEXT,
    active INTEGER NOT NULL DEFAULT 1,
    last_ok_at TEXT,
    consecutive_failures INTEGER NOT NULL DEFAULT 0,
    last_error TEXT,
    PRIMARY KEY (ats, slug)
);
CREATE TABLE IF NOT EXISTS jobs (
    id TEXT PRIMARY KEY,
    source TEXT NOT NULL,
    board_slug TEXT NOT NULL,
    company_name TEXT NOT NULL,
    title TEXT NOT NULL,
    location_raw TEXT,
    remote_flag_raw TEXT,
    department TEXT,
    employment_type_raw TEXT,
    salary_min REAL,
    salary_max REAL,
    salary_currency TEXT,
    description_text TEXT,
    apply_url TEXT NOT NULL,
    posted_at TEXT,
    content_hash TEXT NOT NULL,
    first_seen_at TEXT NOT NULL,
    last_seen_at TEXT NOT NULL,
    expired_at TEXT,
    classify_status TEXT NOT NULL DEFAULT 'pending'
        CHECK (classify_status IN ('pending', 'done', 'skipped', 'error'))
);
CREATE INDEX IF NOT EXISTS jobs_board ON jobs (source, board_slug);
CREATE INDEX IF NOT EXISTS jobs_classify ON jobs (classify_status);
CREATE TABLE IF NOT EXISTS extractions (
    content_hash TEXT PRIMARY KEY,
    model TEXT NOT NULL,
    data TEXT NOT NULL,
    input_tokens INTEGER,
    output_tokens INTEGER,
    created_at TEXT NOT NULL
);
CREATE TABLE IF NOT EXISTS batches (
    name TEXT PRIMARY KEY,
    model TEXT NOT NULL,
    submitted_at TEXT NOT NULL,
    requests INTEGER NOT NULL,
    state TEXT NOT NULL,
    finished_at TEXT
);
CREATE TABLE IF NOT EXISTS runs (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    started_at TEXT NOT NULL,
    finished_at TEXT,
    boards_ok INTEGER DEFAULT 0,
    boards_failed INTEGER DEFAULT 0,
    jobs_new INTEGER DEFAULT 0,
    jobs_updated INTEGER DEFAULT 0,
    jobs_expired INTEGER DEFAULT 0
```

```
);
"""
```

Table	One row per	Key points
`boards`	company board	Health tracking: `consecutive_failures`, `last_error`, `active`. A board failing 5 times in a row is retired.
`jobs`	job ever seen	`first_seen_at`, `last_seen_at`, `expired_at` (NULL while open), `posted_at` (employer's date), `classify_status`: pending, done, skipped or error.
`extractions`	distinct posting content	Keyed by `content_hash`, so identical postings (the same job on several boards) are read by the AI once. Stores the model name and token counts.
`batches`	Gemini batch job	For the paid Batch API mode: name, state, when submitted and finished.
`runs`	fetch run	Totals per run: boards ok and failed, new, updated, expired.
`telegram_posts`	(job, channel)	Created by `telegram.py`: what was already posted where, so nothing is posted twice.

Opening the database also performs small **migrations**: when a column was added later (`posted_at`), `connect()` adds it to older files so CI and local copies keep working.

pipeline/db.py · connect

```
def connect(path: str = config.DB_PATH) -> sqlite3.Connection:
    conn = sqlite3.connect(path)
    conn.row_factory = sqlite3.Row
    conn.executescript(SCHEMA)
    if "posted_at" not in {r[1] for r in conn.execute("PRAGMA table_info(jobs)")}:
        conn.execute("ALTER TABLE jobs ADD COLUMN posted_at TEXT")  # added 2026-10-11
    return conn
```

The five diff rules

Each day the pipeline gets a fresh list of open jobs per board and must update the database. `apply_result` is the heart of it, and implements five rules written down in `docs/SPEC.md`:

1. **Seen again**: refresh `last_seen_at` and clear `expired_at` (a job can come back).
2. **Missing from a successful fetch**: the job is gone, set `expired_at`. Rolling feeds (Jobicy) are the exception: they expire by age.
3. **A failed fetch never expires anything.** If Greenhouse is down for an hour, its jobs must not vanish from the site.
4. **Content changed**: send the job back to the AI (`classify_status = 'pending'`).
5. **Five failures in a row**: retire the board.

pipeline/db.py · apply_result

```
def apply_result(conn: sqlite3.Connection, result: FetchResult, now: str) -> Diff:
    b, diff = result.board, Diff()
    with conn:
        if not result.ok:
            # Rule 3: a failed fetch never expires anything.
            conn.execute(
```

```
            """UPDATE boards SET consecutive_failures = consecutive_failures + 1,
               last_error = ? WHERE ats = ? AND slug = ?""",
            (result.error, b.ats, b.slug),
        )
        # Rule 5: retire a board after N failures in a row.
        cur = conn.execute(
            "UPDATE boards SET active = 0 WHERE ats = ? AND slug = ? AND active = 1 "
            "AND consecutive_failures >= ?",
            (b.ats, b.slug, config.MAX_CONSECUTIVE_FAILURES),
        )
        diff.deactivated = cur.rowcount > 0
        return diff

    conn.execute(
        """UPDATE boards SET consecutive_failures = 0, last_ok_at = ?, last_error = NULL
           WHERE ats = ? AND slug = ?""",
        (now, b.ats, b.slug),
    )
    existing = {
        r["id"]: (r["content_hash"], r["expired_at"], r["since"])
        for r in conn.execute(
            "SELECT id, content_hash, expired_at, COALESCE(posted_at, first_seen_at) since "
            "FROM jobs WHERE source = ? AND board_slug = ?",
            (b.ats, b.slug),
        )
    }
    seen = set()
    for job in result.jobs:
        if job.id in seen:
            continue
        seen.add(job.id)
        values = [getattr(job, f) for f in JOB_FIELDS]
        if job.id not in existing:
            conn.execute(
                f"""INSERT INTO jobs (id, {", ".join(JOB_FIELDS)}, first_seen_at, last_seen_at)
                    VALUES (?, {", ".join("?" * len(JOB_FIELDS))}, ?, ?)""",
                [job.id, *values, now, now],
            )
            diff.new += 1
            continue
        old_hash, expired_at, _ = existing[job.id]
        changed = old_hash != job.content_hash
        # Rule 1: refresh last_seen_at and clear expired_at.
        # Rule 4: a content change sends the job back to the classifier.
        conn.execute(
            f"""UPDATE jobs SET {", ".join(f"{f} = ?" for f in JOB_FIELDS)},
                last_seen_at = ?, expired_at = NULL,
                classify_status = CASE WHEN ? THEN 'pending' ELSE classify_status END
                WHERE id = ?""",
            [*values, now, changed, job.id],
        )
        if changed or expired_at:
            diff.updated += 1
    # Rule 2: anything not in a successful fetch is gone. Rolling feeds: once old enough.
    gone = [i for i, (_, exp, _) in existing.items() if i not in seen and exp is None]
    if days := config.ROLLING_DAYS.get(b.ats):
        cutoff = (datetime.fromisoformat(now) - timedelta(days=days)).isoformat()
        gone = [i for i in gone if existing[i][2] < cutoff]
    conn.executemany("UPDATE jobs SET expired_at = ? WHERE id = ?", [(now, i) for i in gone])
    diff.expired = len(gone)
return diff
```

Reading the code

- `with conn:` opens a **transaction**: either every change for this board is saved, or none is (if an error happens halfway).
- `existing` maps each known job id to its hash, expiry and "since" date, loaded in one query instead of one per job.
- `seen` guards against feeds that list the same job twice.
- The `CASE WHEN ? THEN 'pending' ELSE classify_status END` trick resets the status only when the content changed, in the same UPDATE.

Retiring boards removed from the list

pipeline/db.py · sync_boards

```
def sync_boards(conn: sqlite3.Connection, boards: list[Board], now: str = "") -> int:
    """Add boards from the seed list. Existing rows keep their health state.

    Boards no longer in the seed list are retired and their open jobs expired:
    a deliberate removal, unlike a fetch failure. Returns how many were retired.
    """
    keep = {(b.ats, b.slug) for b in boards}
    with conn:
        gone = [
            (r["ats"], r["slug"])
            for r in conn.execute("SELECT ats, slug FROM boards WHERE active = 1")
            if (r["ats"], r["slug"]) not in keep
        ]
        for ats, slug in gone:
            conn.execute(
                "UPDATE boards SET active = 0, last_error = 'removed from seed list' "
                "WHERE ats = ? AND slug = ?",
                (ats, slug),
            )
            conn.execute(
                "UPDATE jobs SET expired_at = ? WHERE source = ? AND board_slug = ? "
                "AND expired_at IS NULL",
                (now, ats, slug),
            )
        conn.executemany(
            """INSERT INTO boards (ats, slug, company_name, website, source_of_entry)
               VALUES (?, ?, ?, ?, ?)
               ON CONFLICT (ats, slug) DO UPDATE SET company_name = excluded.company_name,
                 website = excluded.website, source_of_entry = excluded.source_of_entry""",
            [(b.ats, b.slug, b.company_name, b.website, b.source_of_entry) for b in boards],
        )
    return len(gone)
```

A board removed from `boards.csv` on purpose is different from a board that fails: its jobs are expired immediately. `ON CONFLICT ... DO UPDATE` is an **upsert**: insert a new board, or update the name and website of an existing one while keeping its health history.

Where the database lives

`jobs.db` is never committed to git (it is about 170 MB and holds full job descriptions). The daily workflow downloads it from the `data-latest` GitHub Release, works on it, and uploads it back, encrypted with AES-256. Chapter 18 shows how.

Chapter 7

Fetching jobs: HTTP, concurrency, connectors

The fetch step downloads the public job feeds of 537 boards in about a minute, turns each into a list of `Job` objects, and applies the diff rules. It must be fast, polite to the companies' servers, and robust to any feed failing.

A polite HTTP client (pipeline/http.py)

pipeline/http.py · get_json

```
async def get_json(client: httpx.AsyncClient, url: str, params: dict | None = None):
    """GET and decode JSON. Retries timeouts, 429 and 5xx. Raises FetchError otherwise."""
    last = ""
    for attempt in range(config.RETRIES + 1):
        if attempt:
            await asyncio.sleep(2**attempt)
        try:
            r = await client.get(url, params=params)
        except httpx.HTTPError as e:
            last = f"{type(e).__name__}: {e}"
            continue
        if r.status_code == 429 or r.status_code >= 500:
            last = f"HTTP {r.status_code}"
            continue
        if r.status_code >= 400:
            raise FetchError(f"HTTP {r.status_code}")
        try:
            return r.json()
        except ValueError as e:
            raise FetchError(f"bad JSON: {e}") from e
    raise FetchError(last)
```

- **Retries with backoff.** A timeout, an HTTP 429 (too many requests) or a 5xx server error is often temporary, so it is retried after 2, then 4 seconds (`2**attempt`). A 4xx error such as 404 is not retried: the board really does not exist.
- **One error type.** Every failure becomes a `FetchError`, so the caller has one thing to catch and a short message to store in `boards.last_error`.
- The client itself (`make_client`) sets the honest User-Agent, the timeout and `follow_redirects`.

Five at a time (pipeline/run.py)

pipeline/run.py · fetch_board

```
async def fetch_board(client, board: Board, sem: asyncio.Semaphore) -> FetchResult:
    async with sem:
        try:
            jobs = await CONNECTORS[board.ats].fetch(client, board)
            return FetchResult(board=board, ok=True, jobs=jobs)
        except (FetchError, KeyError, TypeError, ValueError) as e:
            return FetchResult(board=board, ok=False, error=f"{type(e).__name__}: {e}"[:300])
```

pipeline/run.py · fetch_all

```
async def fetch_all(boards: list[Board]):
    """Yield results as they complete, max MAX_CONCURRENCY in flight."""
    sem = asyncio.Semaphore(config.MAX_CONCURRENCY)
```

```
    async with make_client() as client:
        tasks = [asyncio.create_task(fetch_board(client, b, sem)) for b in boards]
        for done in asyncio.as_completed(tasks):
            yield await done
```

> **asyncio in three sentences**
>
> Most of fetching is waiting for servers to answer. With `asyncio`, a single Python thread starts a request, and while waiting it starts others. A **semaphore** is a counter that lets at most N tasks inside a block at once (here five), and `as_completed` hands back each board's result the moment it finishes, so the database is updated while other downloads are still running.

`fetch_board` never lets an exception escape: a broken feed becomes a `FetchResult(ok=False)`, which rule 3 handles (no expiry) and rule 5 counts (retire after five).

The fetch command

pipeline/run.py · cmd_fetch

```
async def cmd_fetch(conn, limit: int | None) -> str:
    retired = db.sync_boards(conn, load_boards(), now())
    if retired:
        print(f"  retired {retired} boards no longer in the seed list", file=sys.stderr)
    boards = db.active_boards(conn)[:limit] if limit else db.active_boards(conn)
    run_id = db.start_run(conn, now())
    t = {"boards_ok": 0, "boards_failed": 0, "new": 0, "updated": 0, "expired": 0}
    failures, deactivated = [], []
    i = 0
    async for res in fetch_all(boards):
        i += 1
        diff = db.apply_result(conn, res, now())
        t["boards_ok" if res.ok else "boards_failed"] += 1
        t["new"] += diff.new
        t["updated"] += diff.updated
        t["expired"] += diff.expired
        if not res.ok:
            failures.append(f"{res.board.ats}/{res.board.slug}: {res.error}")
        if diff.deactivated:
            deactivated.append(f"{res.board.ats}/{res.board.slug}")
        if i % 50 == 0:
            print(f"  {i}/{len(boards)} boards", file=sys.stderr)
    db.finish_run(conn, run_id, now(), t)

    lines = [
        f"## {config.PRODUCT_NAME} fetch run #{run_id}",
        "",
        "| boards ok | boards failed | new | updated | expired |",
        "|---|---|---|---|---|",
        f"| {t['boards_ok']} | {t['boards_failed']} | {t['new']} | {t['updated']} "
        f"| {t['expired']} |",
    ]
    if deactivated:
        lines += ["", f"**Deactivated after {config.MAX_CONSECUTIVE_FAILURES} failures:**"]
        lines += [f"- {d}" for d in deactivated]
    if failures:
        lines += ["", "<details><summary>Failed boards</summary>", ""]
        lines += [f"- {f}" for f in sorted(failures)] + ["", "</details>"]
    return "\n".join(lines)
```

It returns a Markdown report. Every command does this, and `main()` appends the report to `$GITHUB_STEP_SUMMARY`, so each workflow run shows a readable summary page on GitHub.

Connectors: one module per source

Each connector exposes two functions: `parse(data, board)`, a **pure function** that turns the feed's JSON into `Job` objects (easy to test with saved example feeds), and `async fetch(client, board)`, which downloads and calls parse. A registry maps the ATS name to its module:

pipeline/connectors/__init__.py

```
"""One module per hiring system. Each exposes parse(data, board) and async fetch(client, board)."""

from pipeline.connectors import (
    ashby,
    greenhouse,
    jobicy,
    lever,
    recruitee,
    remoteok,
    smartrecruiters,
    teamtailor,
    workable,
)

CONNECTORS = {
    "greenhouse": greenhouse,
    "lever": lever,
    "ashby": ashby,
    "smartrecruiters": smartrecruiters,
    "recruitee": recruitee,
    "teamtailor": teamtailor,
    "workable": workable,
    "remoteok": remoteok,
    "jobicy": jobicy,
}
```

Greenhouse

pipeline/connectors/greenhouse.py · parse

```
def parse(data: dict, board: Board) -> list[Job]:
    jobs = []
    for j in data["jobs"]:
        depts = j.get("departments") or []
        jobs.append(
            Job(
                source="greenhouse",
                board_slug=board.slug,
                source_job_id=str(j["id"]),
                company_name=board.company_name,
                title=j["title"].strip(),
                location_raw=((j.get("location") or {}).get("name") or "").strip(),
                department=depts[0]["name"] if depts else None,
                employment_type_raw=_meta(j, "employment type"),
                description_text=html_to_text(j.get("content"), escaped=True),
                apply_url=j["absolute_url"],
                posted_at=iso_date(j.get("first_published")),
            )
        )
    return jobs
```

Greenhouse returns all jobs with `?content=true`. Its descriptions are HTML that is itself entity-escaped (`<p>`), hence `html_to_text(..., escaped=True)`. Optional metadata (employment type) is read with a small helper. `first_published` becomes `posted_at`.

Lever

pipeline/connectors/lever.py · parse

```
def parse(data: list, board: Board) -> list[Job]:
    jobs = []
    for j in data:
        cat = j.get("categories") or {}
        locs = cat.get("allLocations") or [cat.get("location") or ""]
        sections = [j.get("descriptionPlain") or ""]
        for item in j.get("lists") or []:
            sections.append(f"{item.get('text', '')}\n{html_to_text(item.get('content'))}")
        sections.append(j.get("additionalPlain") or "")
        sal = j.get("salaryRange") or {}
        jobs.append(
            Job(
                source="lever",
                board_slug=board.slug,
                source_job_id=j["id"],
                company_name=board.company_name,
                title=j["text"].strip(),
                location_raw="; ".join(x for x in locs if x),
                remote_flag_raw=j.get("workplaceType"),
                department=cat.get("department") or cat.get("team"),
                employment_type_raw=cat.get("commitment"),
                salary_min=sal.get("min"),
                salary_max=sal.get("max"),
                salary_currency=sal.get("currency"),
                description_text="\n\n".join(s.strip() for s in sections if s.strip()),
                apply_url=j["hostedUrl"],
                posted_at=iso_date(j.get("createdAt")),
            )
        )
    return jobs
```

Lever gives a plain-text description plus "lists" (Requirements, What you'll do) and a salary range. `workplaceType` (remote, hybrid, onsite) becomes `remote_flag_raw`, which the pre-filter uses. `createdAt` is a Unix time in milliseconds.

Ashby

pipeline/connectors/ashby.py · parse

```
def parse(data: dict, board: Board) -> list[Job]:
    jobs = []
    for j in data["jobs"]:
        if j.get("isListed") is False:
            continue
        locs = [j.get("location") or ""] + [
            s.get("location") or "" for s in j.get("secondaryLocations") or []
        ]
        comp = (j.get("compensation") or {}).get("summaryComponents") or []
        salary = next((c for c in comp if c.get("compensationType") == "Salary"), {})
        remote = j.get("workplaceType") or ("Remote" if j.get("isRemote") else None)
        jobs.append(
            Job(
                source="ashby",
                board_slug=board.slug,
                source_job_id=j["id"],
                company_name=board.company_name,
                title=j["title"].strip(),
                location_raw="; ".join(x for x in locs if x),
                remote_flag_raw=remote,
                department=j.get("department") or j.get("team"),
                employment_type_raw=j.get("employmentType"),
```

```
                salary_min=salary.get("minValue"),
                salary_max=salary.get("maxValue"),
                salary_currency=salary.get("currencyCode"),
                description_text=j.get("descriptionPlain")
                or html_to_text(j.get("descriptionHtml")),
                apply_url=j["jobUrl"],
                posted_at=iso_date(j.get("publishedAt")),
            )
        )
    return jobs
```

Ashby has a secondary-locations list and structured compensation. Unlisted jobs are skipped.

SmartRecruiters: pagination

pipeline/connectors/smartrecruiters.py · fetch

```
async def fetch(client, board: Board) -> list[Job]:
    jobs: list[Job] = []
    url = URL.format(slug=board.slug)
    for page in range(MAX_PAGES):
        data = await get_json(client, url, {"limit": PAGE, "offset": page * PAGE})
        jobs += parse(data, board)
        if (page + 1) * PAGE >= data.get("totalFound", 0):
            return jobs
    raise FetchError(f"more than {MAX_PAGES * PAGE} postings")
```

SmartRecruiters returns 100 postings per page and no description in the list. The connector pages through all of them (needed so expiry is correct) but stops at 10 pages: bigger boards are large on-site employers and a poor fit.

Workable, Recruitee, Teamtailor

Workable's public widget API returns every job with its description in one request; Recruitee's offers API has explicit remote/hybrid/on-site flags; Teamtailor publishes a JSON Feed whose items carry a schema.org `JobPosting`, including `jobLocationType: TELECOMMUTE` for remote jobs.

pipeline/connectors/teamtailor.py · parse

```
def parse(data: dict, board: Board) -> list[Job]:
    jobs = []
    for it in data["items"]:
        jp = it.get("_jobposting") or {}
        places = jp.get("jobLocation") or []
        if isinstance(places, dict):
            places = [places]
        remote = jp.get("jobLocationType") == "TELECOMMUTE"
        req = jp.get("applicantLocationRequirements") or []
        if isinstance(req, dict):
            req = [req]
        location = "; ".join(
            [_place(p) for p in places if _place(p)]
            + [r.get("name", "") for r in req if r.get("name")]
        )
        emp = jp.get("employmentType")
        jobs.append(
            Job(
                source="teamtailor",
                board_slug=board.slug,
                source_job_id=str(it["id"]),
                company_name=board.company_name,
                title=it["title"].strip(),
```

```
                location_raw=("Remote; " if remote else "") + location
                if remote or location
                else "",
                remote_flag_raw="remote" if remote else None,
                employment_type_raw=", ".join(emp) if isinstance(emp, list) else emp,
                description_text=html_to_text(it.get("content_html")),
                apply_url=it["url"],
                posted_at=iso_date(jp.get("datePosted") or it.get("date_published")),
            )
        )
    return jobs
```

Remote OK and Jobicy: job boards with terms

These are not company systems but public job boards whose API terms allow reuse *with a credit and a link back*. Their `apply_url` is the board's own page, and the site names them as the source. Jobicy's feed only covers the last 7 days and pages with a cursor:

pipeline/connectors/jobicy.py · fetch

```
async def fetch(client, board: Board) -> list[Job]:
    jobs: list[Job] = []
    params = {"count": 100}
    for _ in range(MAX_PAGES):
        data = await get_json(client, URL, params)
        jobs += parse(data, board)
        if not data.get("hasMore") or not data.get("nextCursor"):
            break
        params = {"count": 100, "cursor": data["nextCursor"]}
    return jobs
```

> **The legal line**
>
> HiresHere only reads feeds a company publishes for its own careers page, or board APIs whose terms allow reuse. It never scrapes LinkedIn, Indeed, Glassdoor or any site whose terms forbid it, and never republishes full descriptions on the website (only short quotes and fields). Remotive and Himalayas were left out because their terms ask not to feed other job sites.

Chapter 8

Cleaning text and dates

Every feed formats things differently. Two small helpers in `pipeline/text.py` turn HTML descriptions into readable text and any date format into one standard form.

HTML to text with the standard library

Instead of a heavy HTML library, a subclass of Python's built-in `HTMLParser` collects text and inserts line breaks around block elements:

pipeline/text.py · _Stripper

```
class _Stripper(HTMLParser):
    def __init__(self):
        super().__init__(convert_charrefs=True)
        self.parts: list[str] = []
        self.skip = 0

    def handle_starttag(self, tag, attrs):
        if tag in ("script", "style"):
            self.skip += 1
        elif tag in _BLOCK:
            self.parts.append("\n")
            if tag == "li":
                self.parts.append("- ")

    def handle_endtag(self, tag):
        if tag in ("script", "style"):
            self.skip = max(0, self.skip - 1)
        elif tag in _BLOCK:
            self.parts.append("\n")

    def handle_data(self, data):
        if not self.skip:
            self.parts.append(data)
```

pipeline/text.py · html_to_text

```
def html_to_text(raw: str | None, escaped: bool = False) -> str:
    """Strip tags. Set escaped=True when the HTML itself is entity-escaped (Greenhouse)."""
    if not raw:
        return ""
    if escaped:
        raw = html.unescape(raw)
    p = _Stripper()
    p.feed(raw)
    p.close()
    text = "".join(p.parts).replace("\xa0", " ")
    text = re.sub(r"[ \t]+", " ", text)
    text = re.sub(r" *\n *", "\n", text)
    return re.sub(r"\n{3,}", "\n\n", text).strip()
```

- Script and style contents are dropped (`self.skip` counts nesting).
- Block tags (`p`, `div`, `li`…) become line breaks; list items get a "- " prefix, so the AI still sees the structure.
- Regular expressions then collapse spaces and keep at most one blank line.

Dates from any feed

pipeline/text.py · iso_date

```
def iso_date(v) -> str | None:
    """A posting date from any feed (ISO text, "... UTC", epoch ms) as ISO 8601 UTC, or None."""
    if v in (None, ""):
        return None
    try:
        if isinstance(v, int | float) or str(v).isdigit():
            d = datetime.fromtimestamp(int(v) / 1000, UTC)
        else:
            d = datetime.fromisoformat(str(v).replace(" UTC", "+00:00").replace("Z", "+00:00"))
            d = d.replace(tzinfo=d.tzinfo or UTC).astimezone(UTC)
    except (ValueError, OverflowError, OSError):
        return None
    return d.replace(microsecond=0).isoformat()
```

Feeds send ISO dates with offsets (`2026-08-17T14:44:07-04:00`), with a trailing `Z`, with " UTC", as a plain date, or as milliseconds since 1970 (Lever). Everything becomes an ISO 8601 string in UTC, so dates sort correctly as text and compare across sources. Bad input returns `None` rather than crashing the fetch.

> **Try it**
>
> In a Python shell from the repository: `from pipeline.text import iso_date`, then `iso_date(1782214185805)` and `iso_date("2026-10-05 16:05:18 UTC")`. The test file `tests/test_text.py` lists every format it must handle.

Why the employer's date matters

Before `posted_at` existed, the site showed "found 2 days ago", the date HiresHere first saw a job. The first real fetch showed that 47% of open jobs had been posted more than two months earlier (some in 2016). Now cards say "Posted n days ago", sorting uses that date, postings older than 60 days are drawn muted, and alerts and Telegram only announce jobs posted in the last 14 days.

Chapter 9

The company list and the logos

HiresHere can only show jobs from companies it knows. `pipeline/seed.py` builds and validates the list of boards; `pipeline/logos.py` finds each company's website and icon.

Where the companies come from

Two open lists, both credited in `data/CREDITS.md`:

- **remoteintech/remote-jobs** (ISC licence): a curated list of remote-friendly companies with their careers page.
- **kalilo321/ats-scrapers** (MIT licence): CSV files mapping thousands of companies to their slug on each ATS.

`seed build` joins them: first it reads the careers URL (a `jobs.lever.co/acme` link reveals ATS and slug directly), then matches the company name against the ATS lists, and finally guesses slugs with one cheap request each.

pipeline/seed.py · probe

```
async def probe(client, sem, ats: str, slug: str) -> bool:
    url, params = PROBE[ats]
    async with sem:
        try:
            data = await get_json(client, url.format(s=slug), params)
        except FetchError:
            return False
    if ats == "smartrecruiters":
        return bool(data.get("totalFound"))
    if ats == "lever":
        return isinstance(data, list) and bool(data)
    return isinstance(data, dict) and "error" not in data
```

Validation and the noise filter

`seed validate` fetches every candidate once and keeps those that answer and have jobs. Boards added from the big inventories (not hand-curated) must also pass `noise()`: at least 5% of their jobs must look remote, or they must have two or more entry-level openings that are not office-only. Template spam (hundreds of jobs with a handful of titles) is dropped too.

pipeline/seed.py · entry_level

```
def entry_level(jobs: list) -> int:
    """Intern, junior and graduate openings not flagged office-only."""
    return sum(
        bool(ENTRY_LEVEL.search(j.title))
        and re.sub(r"[\s-]", "", (j.remote_flag_raw or "").lower()) not in OFFICE_FLAGS
        for j in jobs
    )
```

pipeline/seed.py · noise

```
def noise(jobs: list) -> str | None:
    """Why a top-up board is not worth fetching, or None. Jobs need location_raw,
    remote_flag_raw and title. Only applied to ats-scrapers top-ups: remoteintech
```

```
    companies are curated, and their location text often omits "remote". A board with
    a few entry-level openings is kept even when few jobs say remote."""
    if not jobs:
        return "no open jobs"
    remote = sum(bool(REMOTE.search(f"{j.location_raw} {j.remote_flag_raw or ''}")) for j in jobs)
    if remote / len(jobs) < 0.05 and entry_level(jobs) < 2:
        return f"only {remote}/{len(jobs)} remote-looking jobs"
    if len(jobs) >= 50 and len({j.title for j in jobs}) / len(jobs) < 0.10:
        return f"template spam: {len({j.title for j in jobs})} titles for {len(jobs)} jobs"
    return None
```

Choosing companies for juniors

In October 2026, 162 companies were added from a random sample of 2,800 boards, chosen for intern, junior and graduate openings. After the first classification run, the 44 whose jobs were all office-only were removed. `source_of_entry = ats-scrapers+entry` marks them in `boards.csv`.

Logos (pipeline/logos.py)

For each company with a website, the script downloads the page, parses its `<link rel="icon">` tags and picks the best one (an `apple-touch-icon` first, then the largest declared size). Logos are saved under `site/public/logos/` and committed, so pages never load images from third-party sites.

pipeline/logos.py · Icons

```
class Icons(HTMLParser):
    def __init__(self):
        super().__init__()
        self.found: list[tuple[int, str]] = []

    def handle_starttag(self, tag, attrs):
        a = dict(attrs)
        rel = (a.get("rel") or "").lower()
        if tag != "link" or "icon" not in rel or not a.get("href") or a["href"].startswith("data:"):
            return
        size = max(
            (int(s.split("x")[0]) for s in (a.get("sizes") or "").split() if "x" in s), default=0
        )
        rank = (
            1000
            if "apple-touch-icon" in rel
            else size or (64 if a["href"].endswith(".svg") else 16)
        )
        self.found.append((rank, a["href"]))
```

Many boards have no website in the lists, so the script first reads it from the company's own board page (each ATS shows a "home page" link in a known place), rejecting links that point to a hiring system or a placeholder domain:

pipeline/logos.py · HOME

```
HOME = {
    "ashby": ("https://jobs.ashbyhq.com/{}", r'"publicWebsite":"(https?://[^"]+)"'),
    "lever": (
        "https://jobs.lever.co/{}",
        r'class="main-footer-text"><p><a href="(https?://[^"]+)"',
    ),
    "teamtailor": (
        "https://{}.teamtailor.com/",
        r'Homepage</span>\s*<a[^>]*href="(https?://[^"]+)"',
```

```
    ),
    "smartrecruiters": (
        "https://careers.smartrecruiters.com/{}",
        r'<li class="nav-item"><a href="(https?://[^"]+)" class="nav-link"',
    ),
}
```

A `SKIP` set lists icons that came back unusable (white on white, empty, corrupt), so those companies show their initials on a coloured tile instead.

Chapter 10

Classification: asking an AI the right question

The AI's only job is to read one posting and fill a strict form: where the person may live, which time zones, work authorization, seniority, and the sentence that proves it. Getting this step right is mostly about a precise prompt, a strict schema, and checking the answer.

Step 1: the pre-filter (no AI)

About half of all open jobs are clearly office-based. Sending them to the AI would waste free quota, so a regular-expression pre-filter skips them first:

pipeline/classify/prompt.py · lines 43 to 55

```
REMOTE_SIGNAL = re.compile(
    r"remote|anywhere|worldwide|global|emea|africa|\bmena\b|distributed|home[- ]based"
    r"|work from home|telecommut",
    re.I,
)
DESC_REMOTE = re.compile(r"\bremote\b|work from anywhere|fully distributed", re.I)
# Entry-level postings often name only a city; let the model read them like any other level.
ENTRY_LEVEL = re.compile(
    r"\b(intern(ship)?s?|stage|stagiaire|trainee|graduate|new grad|junior|jr|entry[- ]level"
    r"|werkstudent|working student|apprentice(ship)?|alternan(ce|t)|student)\b",
    re.I,
)
OFFICE_FLAGS = {"onsite", "inoffice"}
```

pipeline/classify/prompt.py · prefilter

```
def prefilter(job: dict) -> str | None:
    """Reason to skip without calling the model, or None to classify.

    Conservative: skip only when nothing in the structured fields or text hints at remote.
    """
    head = f"{job['location_raw'] or ''} {job['title']}"
    flag = re.sub(r"[\s-]", "", (job["remote_flag_raw"] or "").lower())
    if flag in ("remote", "hybrid") or REMOTE_SIGNAL.search(head):
        return None
    if flag in OFFICE_FLAGS:
        return f"workplace flag {flag}"
    if ENTRY_LEVEL.search(job["title"]) or DESC_REMOTE.search(job["description_text"] or ""):
        return None
    return "no remote signal"
```

It is **conservative**: a job is skipped only when nothing hints at remote work. A remote or hybrid flag, a remote word in the title or location, an entry-level title (interns often list only a city), or "remote" in the description all send it to the AI. An explicit office-only flag skips it. Skipped jobs are re-checked every run, so improving the pre-filter applies to old jobs too.

Step 2: the prompt

The system prompt defines every field, with examples and the traps found during testing. The most important rule: **the location field is authoritative**; company slogans like "we hire globally" do not make a US-only role worldwide.

pipeline/classify/prompt.py · lines 5 to 12

```
SYSTEM = """You read one job posting and extract who is allowed to apply for it.
Report only what the posting states or clearly implies. Never guess. When the posting is silent, use "unclear", an
empty list or null.

Fields:
- remote_type: fully_remote (no office attendance), hybrid (some office days), onsite, unclear.
- geo_scope: where the candidate may live and work from.
  worldwide = this role is explicitly open to candidates in any country ("Remote - Worldwide", "we hire in any
country", "you can be located anywhere in the world").
  regions = limited to named regions (EMEA, Europe, LATAM...).
```

Each posting is sent as a short, labelled message. Long descriptions are cut down by `excerpt()` to the opening plus every paragraph that mentions eligibility words (location, visa, time zone, contract…), at most 4,000 characters:

pipeline/classify/prompt.py · user_message

```
def user_message(job: dict) -> str:
    return (
        f"Company: {job['company_name']}\n"
        f"Title: {job['title']}\n"
        f"Location field: {job['location_raw'] or '(empty)'}\n"
        f"Workplace flag: {job['remote_flag_raw'] or '(none)'}\n"
        f"Employment type: {job['employment_type_raw'] or '(none)'}\n\n"
        f"Posting excerpt:\n{excerpt(job['description_text'] or '')}"
    )
```

pipeline/classify/prompt.py · excerpt

```
def excerpt(text: str, head: int = 1200, limit: int = 4000) -> str:
    """Opening of the posting plus every paragraph that mentions eligibility keywords."""
    if len(text) <= limit:
        return text
    out, size = [text[:head]], head
    for para in re.split(r"\n+", text[head:]):
        para = para.strip()
        if para and KEYWORDS.search(para) and size + len(para) <= limit:
            out.append(para)
            size += len(para) + 1
    return "\n".join(out)
```

Step 3: the schema

The answer must match the `Extraction` model exactly. Validators clean it: country codes are upper-cased and filtered to two letters (and "UK" becomes "GB"), regions must come from a fixed list, the quote is trimmed to 200 characters.

pipeline/classify/schema.py · Extraction

```
class Extraction(BaseModel):
    remote_type: Literal["fully_remote", "hybrid", "onsite", "unclear"]
    geo_scope: Literal["worldwide", "regions", "countries", "timezone_only", "unclear"]
    allowed_countries: list[str] = []
    excluded_countries: list[str] = []
    allowed_regions: list[str] = []
    timezone_requirement: str | None = None
    timezone_utc_min: float | None = None
    timezone_utc_max: float | None = None
    work_authorization_required: list[str] = []
```

```
    engagement: Literal["employee", "contractor", "either", "unclear"]
    languages_required: list[str] = []
    seniority: Literal["intern", "junior", "mid", "senior", "lead", "unclear"]
    role_family: Literal[
        "engineering", "data", "design", "product", "marketing", "sales", "support", "ops", "other"
    ]
    evidence_quote: str
    confidence: Literal["high", "medium", "low"]

    @field_validator("allowed_countries", "excluded_countries", "work_authorization_required")
    @classmethod
    def _iso2(cls, v: list[str]) -> list[str]:
        out = []
        for c in v:
            c = c.strip().upper()
            c = {"UK": "GB"}.get(c, c)
            if re.fullmatch(r"[A-Z]{2}", c) and c not in out:
                out.append(c)
        return out

    @field_validator("allowed_regions")
    @classmethod
    def _regions(cls, v: list[str]) -> list[str]:
        return [r for r in v if r in REGIONS]

    @field_validator("languages_required")
    @classmethod
    def _langs(cls, v: list[str]) -> list[str]:
        return [x.strip().lower() for x in v if re.fullmatch(r"[a-zA-Z]{2}", x.strip())]

    @field_validator("timezone_requirement")
    @classmethod
    def _tz(cls, v: str | None) -> str | None:
        return v.strip() or None if v else None

    @field_validator("evidence_quote")
    @classmethod
    def _quote(cls, v: str) -> str:
        return v.strip()[:200]
```

The same structure is written out as a JSON Schema, sent with every request, so the model is **forced** to answer in that shape (Gemini's `responseJsonSchema`, OpenAI-style `response_format: json_schema`):

pipeline/classify/schema.py · JSON_SCHEMA

```
JSON_SCHEMA = {
    "type": "object",
    "properties": {
        "remote_type": _enum("fully_remote", "hybrid", "onsite", "unclear"),
        "geo_scope": _enum("worldwide", "regions", "countries", "timezone_only", "unclear"),
        "allowed_countries": _arr(),
        "excluded_countries": _arr(),
        "allowed_regions": _arr(_enum(*REGIONS)),
        "timezone_requirement": {"type": ["string", "null"]},
        "timezone_utc_min": {"type": ["number", "null"]},
        "timezone_utc_max": {"type": ["number", "null"]},
        "work_authorization_required": _arr(),
        "engagement": _enum("employee", "contractor", "either", "unclear"),
        "languages_required": _arr(),
        "seniority": _enum("intern", "junior", "mid", "senior", "lead", "unclear"),
        "role_family": _enum(
            "engineering", "data", "design", "product", "marketing", "sales", "support", "ops",
            "other",
        ),
        "evidence_quote": {"type": "string"},
        "confidence": _enum("high", "medium", "low"),
    },
```

```
    "required": list(Extraction.model_fields),
}
```

> **Structured output**
>
> Without a schema, a model might answer in prose, add fields, or misspell a value. With one, the provider constrains the generation so the output parses as JSON with the expected keys and enum values. Pydantic then validates it again on our side: belt and braces.

Step 4: checking the evidence

Models sometimes paraphrase or invent a quote. Every saved extraction is checked: the quote, normalised (lower case, punctuation removed), must appear in the posting. If not, confidence drops to *low*, and low confidence can never produce a firm "You can apply" (chapter 12).

pipeline/classify/runner.py · verify_evidence

```
def verify_evidence(e: Extraction, job: dict) -> bool:
    """The quote must really appear in the posting. If not, confidence drops to low."""
    source = _norm(
        " ".join(str(job[k] or "") for k in ("title", "location_raw", "description_text"))
    )
    quote = re.sub(
        r"^(location field|workplace flag|title)\s*:\s*", "", e.evidence_quote, flags=re.I
    )
    ok = bool(quote) and _norm(quote) in source
    if not ok:
        e.confidence = "low"
    return ok
```

Step 5: choosing what to send (triage)

pipeline/classify/runner.py · triage

```
def triage(conn: sqlite3.Connection, limit: int | None = None) -> tuple[list[dict], dict]:
    """Mark pre-filtered jobs skipped and reuse known extractions. Returns one job per
    content_hash that still needs the model, plus counts."""
    rows = conn.execute(
        f"SELECT {COLS} FROM jobs WHERE expired_at IS NULL "
        # Newest first: fresh jobs matter most (alerts, Telegram); the backlog fills spare capacity.
        "AND classify_status IN ('pending', 'error', 'skipped') ORDER BY first_seen_at DESC, id"
    ).fetchall()
    known = {r[0] for r in conn.execute("SELECT content_hash FROM extractions")}
    todo: dict[str, dict] = {}
    counts = {"skipped": 0, "reused": 0}
    with conn:
        for r in rows:
            job = dict(r)
            if prefilter(job):
                conn.execute(
                    "UPDATE jobs SET classify_status = 'skipped' WHERE id = ?", (job["id"],)
                )
                counts["skipped"] += 1
            elif job["content_hash"] in known:
                conn.execute("UPDATE jobs SET classify_status = 'done' WHERE id = ?", (job["id"],))
                counts["reused"] += 1
            else:
                todo.setdefault(job["content_hash"], job)
    jobs = list(todo.values())
    return (jobs[:limit] if limit else jobs), counts
```

- **Newest first** (`ORDER BY first_seen_at DESC`): with a limited free quota, today's jobs matter most for alerts and Telegram.
- **Reuse**: if an extraction with the same `content_hash` already exists (the same posting on two boards, or an unchanged repost), it is marked done without calling the AI.
- **One request per distinct content** (`todo.setdefault(job["content_hash"], job)`).

Chapter 11

Running the AI on free tiers

The same extraction can be produced by several providers. Since October 2026 the owner has no AI budget, so HiresHere runs Gemini, Groq and Mistral side by side on their free tiers, each paced under its limits, all pulling from one shared queue.

Talking to Gemini (pipeline/classify/gemini.py)

This is the only file that knows Gemini's API. A request carries the system prompt, the posting, and the JSON schema; temperature 0 makes answers as repeatable as possible:

pipeline/classify/gemini.py · request_body

```
def request_body(job: dict) -> dict:
    return {
        "systemInstruction": {"parts": [{"text": SYSTEM}]},
        "contents": [{"role": "user", "parts": [{"text": user_message(job)}]}],
        "generationConfig": {
            "responseMimeType": "application/json",
            "responseJsonSchema": JSON_SCHEMA,
            "temperature": 0,
        },
    }
```

pipeline/classify/gemini.py · parse_response

```
def parse_response(data: dict) -> tuple[Extraction, int, int]:
    """generateContent response -> (extraction, input tokens, output tokens)."""
    try:
        text = data["candidates"][0]["content"]["parts"][0]["text"]
    except (KeyError, IndexError) as e:
        raise ModelError(f"no content: {json.dumps(data)[:300]}") from e
    usage = data.get("usageMetadata", {})
    out = usage.get("candidatesTokenCount", 0) + usage.get("thoughtsTokenCount", 0)
    try:
        return Extraction.model_validate_json(text), usage.get("promptTokenCount", 0), out
    except ValueError as e:
        raise ModelError(f"invalid extraction: {e}"[:300]) from e
```

Errors are sorted carefully, because they mean different things:

pipeline/classify/gemini.py · _post

```
async def _post(client: httpx.AsyncClient, url: str, **kw) -> httpx.Response:
    for attempt in range(5):
        if attempt:
            await asyncio.sleep(min(60, 4 * 2**attempt))
        try:
            r = await client.post(url, **kw)
        except httpx.HTTPError:
            continue
        if r.status_code == 429 and "perday" in r.text.lower().replace("_", "").replace(" ", ""):
            raise QuotaError(f"Gemini daily quota: {r.text[:200]}")
        if r.status_code == 402:
            raise QuotaError(f"HTTP 402: {r.text[:200]}")
        if r.status_code == 429 or r.status_code >= 500:
            continue
        if r.status_code >= 400:
            raise ModelError(f"HTTP {r.status_code}: {r.text[:300]}")
```

```
        return r
    raise ModelError(f"gave up after retries: {url}")
```

Answer	Meaning	What the code does
429 with "PerDay" in the body	Free daily quota used up	`QuotaError`: this provider stops for today, the others continue.
402	Prepaid credit exhausted	`QuotaError` as well.
Other 429, or 5xx	Too fast, or a server hiccup	Retry with growing waits (8, 16, 32, 60 seconds).
Other 4xx	A real error with this request	`ModelError`: this posting is marked "error" and retried on a later run.

OpenAI-compatible providers (pipeline/classify/compat.py)

Groq and Mistral both speak the OpenAI "chat completions" format, so one class serves both; each provider is just configuration:

pipeline/classify/compat.py · Provider

```
class Provider:
    def __init__(self, name: str, url: str, key_env: str, model: str, rpm_env: str, extra: dict):
        self.name, self.url, self.key_env, self.model = name, url, key_env, model
        self.rpm_env, self.extra = rpm_env, extra

    def available(self) -> bool:
        return bool(os.environ.get(self.key_env))

    @property
    def rpm(self) -> float:
        return float(os.environ.get(self.rpm_env, "0"))

    def make_client(self) -> httpx.AsyncClient:
        return httpx.AsyncClient(
            headers={
                "Authorization": f"Bearer {os.environ[self.key_env]}",
                "User-Agent": config.USER_AGENT,
            },
            timeout=120,
        )

    def request_body(self, job: dict) -> dict:
        return {
            "model": self.model,
            "messages": [
                {"role": "system", "content": SYSTEM},
                {"role": "user", "content": user_message(job)},
            ],
            "response_format": {
                "type": "json_schema",
                "json_schema": {"name": "extraction", "schema": JSON_SCHEMA, "strict": False},
            },
            "temperature": 0,
            **self.extra,
        }

    async def generate(self, client: httpx.AsyncClient, job: dict) -> tuple[Extraction, int, int]:
        r = None
        for attempt in range(5):
            try:
```

```
                r = await client.post(self.url, json=self.request_body(job))
            except httpx.HTTPError:
                await asyncio.sleep(4 * 2**attempt)
                continue
            if r.status_code == 429:
                # Daily or monthly caps end the run for this provider; per-minute caps say how
                # long to wait.
                body = r.text.lower()
                no_plan = r.headers.get("x-ratelimit-limit-req-minute") == "0"  # e.g. no free plan
                if no_plan or "per day" in body or "per month" in body or "monthly" in body:
                    raise QuotaError(f"{self.name} quota: {r.text[:200]}")
                await asyncio.sleep(min(60, float(r.headers.get("retry-after") or 10)))
                continue
            if r.status_code == 402:
                raise QuotaError(f"{self.name} HTTP 402: {r.text[:200]}")
            if r.status_code >= 500:
                await asyncio.sleep(4 * 2**attempt)
                continue
            if r.status_code >= 400:
                raise ModelError(f"{self.name} HTTP {r.status_code}: {r.text[:300]}")
            return parse_response(r.json())
        if r is not None and r.status_code == 429:
            # Still rate limited after every retry: leave the queue to the other providers.
            raise QuotaError(f"{self.name}: still rate limited after retries")
        raise ModelError(f"{self.name}: gave up after retries")
```

pipeline/classify/compat.py · GROQ

```
GROQ = Provider(
    "Groq",
    "https://api.groq.com/openai/v1/chat/completions",
    "GROQ_API_KEY",
    os.environ.get("GROQ_MODEL", "openai/gpt-oss-120b"),
    "GROQ_RPM",
    {"reasoning_effort": "low"},
)
```

pipeline/classify/compat.py · MISTRAL

```
MISTRAL = Provider(
    "Mistral",
    "https://api.mistral.ai/v1/chat/completions",
    "MISTRAL_API_KEY",
    os.environ.get("MISTRAL_MODEL", "mistral-small-latest"),
    "MISTRAL_RPM",
    {},
)
```

Failing gracefully

A provider whose account allows zero requests a minute (the Mistral case: `x-ratelimit-limit-req-minute: 0`), or that keeps answering 429 after every retry, raises `QuotaError` and leaves the run. Without this, it would mark postings as errors and take them away from the providers that work.

Three workers, one queue (runner.run_sync)

pipeline/classify/runner.py · run_sync

```
async def run_sync(conn: sqlite3.Connection, jobs: list[dict]) -> dict:
    """Free tiers, each provider paced to its own limit and working the same queue at the same
    time: Gemini and Groq. A provider stops at its daily quota; the others carry on. Jobs left
    when time or every quota runs out stay pending for the next run."""
```

```
    from collections import deque

    from pipeline.classify import compat

    providers = [(gemini, config.CLASSIFIER_MODEL, config.GEMINI_RPM)]
    providers += [(p, p.model, p.rpm) for p in compat.EXTRA if p.available()]
    t = {"done": 0, "errors": 0, "input_tokens": 0, "output_tokens": 0, "left": 0, "by": {}}
    queue = deque(jobs)
    loop = asyncio.get_running_loop()
    deadline = loop.time() + config.CLASSIFY_MINUTES * 60 if config.CLASSIFY_MINUTES else None

    async def worker(mod, model: str, rpm: float) -> None:
        async with mod.make_client() as client:
            while queue and not (deadline and loop.time() > deadline):
                job = queue.popleft()
                started = loop.time()
                try:
                    e, tin, tout = await mod.generate(client, job)
                except gemini.QuotaError as err:
                    print(f"  {model}: {err}"[:200], file=sys.stderr)
                    queue.appendleft(job)  # another provider takes it
                    return
                except gemini.ModelError as err:
                    print(f"  error {job['id']}: {err}"[:300], file=sys.stderr)
                    mark_error(conn, job["content_hash"])
                    t["errors"] += 1
                else:
                    save(conn, job, e, tin, tout, model)
                    t["done"] += 1
                    t["by"][model] = t["by"].get(model, 0) + 1
                    t["input_tokens"] += tin
                    t["output_tokens"] += tout
                    if t["done"] % 50 == 0:
                        print(f"  {t['done']}/{len(jobs)} classified", file=sys.stderr)
                if rpm:
                    wait = 60 / rpm - (loop.time() - started)
                    if wait > 0:
                        await asyncio.sleep(wait)

    await asyncio.gather(*(worker(*p) for p in providers))
    t["left"] = len(queue)
    return t
```

- The queue is a `deque` (double-ended queue). Each worker takes the next job from the front with `popleft()`.
- On a quota error the job is put back at the front (`appendleft`) for another provider, and that worker stops.
- **Pacing**: after each request the worker sleeps so that it never exceeds its requests per minute (Gemini 14, Groq 4, Mistral 50 in the workflow).
- **Time budget**: `CLASSIFY_MINUTES = 35`. When it runs out, workers stop taking jobs; the rest stay pending for the next day.
- `asyncio.gather` runs the workers at the same time and waits for all of them.
- Each saved extraction records which model produced it, so results can be audited later.

The classify command

pipeline/run.py · cmd_classify

```
async def cmd_classify(conn, limit: int | None, batch: bool, yes: bool) -> str:
    from pipeline.classify import runner
```

```
    if busy := runner.open_batches(conn):
        return f"A batch is still open ({', '.join(busy)}). Run classify-collect first."
    jobs, counts = runner.triage(conn, limit)
    tin, tout, usd = runner.estimate(jobs, batch)
    mode = "batch" if batch else "sync"
    lines = [
        f"## {config.PRODUCT_NAME} classify ({mode}, {config.CLASSIFIER_MODEL})",
        "",
        f"Pre-filtered as on-site or no remote signal: {counts['skipped']}. "
        f"Reused earlier extractions: {counts['reused']}.",
        f"To classify: {len(jobs)} unique postings, about {tin:,} input and {tout:,} output "
        f"tokens, estimated **${usd:.2f}**.",
    ]
    if not jobs:
        return "\n".join(lines)
    if usd > config.MAX_USD_WITHOUT_CONFIRM and not yes:
        lines.append(f"Over ${config.MAX_USD_WITHOUT_CONFIRM:.0f}: not started. Re-run with --yes.")
        return "\n".join(lines)
    print("\n".join(lines), file=sys.stderr)
    if batch:
        from pipeline.classify.gemini import ModelError

        try:
            name = await runner.submit_batch(conn, jobs)
        except ModelError as e:
            # Out of credit (HTTP 402) must not stop the daily run: everything already
            # classified still exports and deploys, and these postings wait for the next run.
            if "HTTP 402" not in str(e):
                raise
            lines.append(f"Not submitted: the model account is out of credit. {len(jobs)} wait.")
            return "\n".join(lines)
        lines.append(f"Submitted batch `{name}`. Run classify-collect later.")
    else:
        t = await runner.run_sync(conn, jobs)
        by = ", ".join(f"{m}: {n}" for m, n in t["by"].items()) or "none"
        real = (t["input_tokens"] * config.PRICE_IN + t["output_tokens"] * config.PRICE_OUT) / 1e6
        lines.append(
            f"Classified {t['done']} ({by}), "
            f"errors {t['errors']}, left for the next run {t['left']}. Actual tokens: "
            f"{t['input_tokens']:,} in, {t['output_tokens']:,} out (about ${real:.2f} at paid "
            "Gemini prices; free tiers cost nothing)."
        )
    return "\n".join(lines)
```

It reports what was skipped and reused, estimates tokens and cost (from the paid prices, as a reference), and runs either the free sync mode or the paid batch mode. An out-of-credit answer at batch submission does not fail the run: postings simply wait.

The paid Batch API (kept for later)

When there is a budget, batch mode is half price: the pipeline uploads one JSONL file with every request, Gemini processes it within hours, and a later step downloads the results. Each line carries a `key` (the content hash), because results come back keyed, not in order, and individual lines can fail while the batch succeeds.

pipeline/classify/gemini.py · batch_submit

```
async def batch_submit(client: httpx.AsyncClient, lines: list[dict], display_name: str) -> str:
    payload = "\n".join(json.dumps(x) for x in lines).encode()
    start = await _post(
        client,
        f"{BASE}/upload/v1beta/files",
        headers={
            "X-Goog-Upload-Protocol": "resumable",
```

```
            "X-Goog-Upload-Command": "start",
            "X-Goog-Upload-Header-Content-Length": str(len(payload)),
            "X-Goog-Upload-Header-Content-Type": "application/jsonl",
        },
        json={"file": {"display_name": display_name}},
    )
    upload_url = start.headers["x-goog-upload-url"]
    done = await _post(
        client,
        upload_url,
        headers={"X-Goog-Upload-Command": "upload, finalize", "X-Goog-Upload-Offset": "0"},
        content=payload,
    )
    file_name = done.json()["file"]["name"]
    r = await _post(
        client,
        f"{BASE}/v1beta/models/{config.CLASSIFIER_MODEL}:batchGenerateContent",
        json={"batch": {"display_name": display_name, "input_config": {"file_name": file_name}}},
    )
    return r.json()["name"]
```

pipeline/classify/runner.py · collect

```
async def collect(conn: sqlite3.Connection) -> list[str]:
    """Poll open batches once and store finished results. Returns status lines."""
    report = []
    async with gemini.make_client() as client:
        for name in open_batches(conn):
            state, file_name = await gemini.batch_status(client, name)
            if not state.endswith("_SUCCEEDED") or not file_name:
                with conn:
                    conn.execute("UPDATE batches SET state = ? WHERE name = ?", (state, name))
                report.append(f"{name}: {state}")
                continue
            results = await gemini.batch_results(client, file_name)
            hashes = [r.get("key") for r in results]
            jobs = {
                r["content_hash"]: dict(r)
                for r in conn.execute(
                    f"SELECT {COLS} FROM jobs WHERE content_hash IN "
                    f"({','.join('?' * len(hashes))})",
                    hashes,
                )
            }
            ok = err = 0
            for r in results:
                job = jobs.get(r.get("key"))
                if not job:
                    continue
                try:
                    if "error" in r:
                        raise gemini.ModelError(str(r["error"])[:300])
                    e, tin, tout = gemini.parse_response(r["response"])
                    save(conn, job, e, tin, tout)
                    ok += 1
                except gemini.ModelError:
                    mark_error(conn, job["content_hash"])
                    err += 1
            with conn:
                conn.execute(
                    "UPDATE batches SET state = 'COLLECTED', finished_at = ? WHERE name = ?",
                    (now(), name),
                )
            report.append(f"{name}: collected {ok} extractions, {err} errors")
    return report
```

How good are the free models?

A spot check on real postings, comparing with the stored paid Gemini results:

Field	Gemini free	Groq gpt-oss-120b
Allowed countries	8/8	8/8
Remote or hybrid	8/8	7/8
Seniority	8/8	6/8

Gemini's free tier gives the same answers as the paid model; Groq is good enough as a parallel worker. Mistral will be checked the same way once its free plan is active.

Chapter 12

The verdict engine: rules, not AI

`pipeline/classify/verdict.py` answers one question with plain Python: given this extraction and this country, can someone living there apply? It runs 243 times per job at export time, costs nothing, and is fully tested.

The four verdicts

Verdict	On the site	When
`eligible`	"You can apply" (green, filled pill)	The posting names the country, or is explicitly open worldwide with role-level evidence.
`likely`	"Likely open to you" (amber, half pill)	A region that contains the country (EMEA for Tunisia), a time zone window that fits, an office in the country, or a low-confidence "eligible".
`unclear`	"Ask the employer" (grey ring)	Remote, but the posting does not say from where.
`not_eligible`	not shown	Excluded, limited elsewhere, needs work authorization elsewhere, wrong time zones, office elsewhere.

The decision, step by step

The order of the checks matters: the first rule that applies wins, and the strongest "no" signals come first.

pipeline/classify/verdict.py · decide

```
def decide(e: Extraction, code: str, country: dict, location: str = "", title: str = ""):
    """(verdict, reason key, reason params) for one country. Params never name the country."""
    regions = set(country["regions"])

    if code in e.excluded_countries:
        return "not_eligible", "excluded", {}

    if e.remote_type in ("onsite", "hybrid"):
        key = "office_here" if code in e.allowed_countries else "office_elsewhere"
        return ("likely" if key == "office_here" else "not_eligible"), key, {"where": e.remote_type}

    other_auth = [c for c in e.work_authorization_required if c != code]
    if other_auth and code not in e.work_authorization_required:
        return "not_eligible", "auth", {"places": ", ".join(other_auth)}

    if code in e.allowed_countries:
        return _cap(e, "eligible"), "listed", {}

    if e.geo_scope == "countries" and e.allowed_countries:
        return "not_eligible", "countries_only", {"places": ", ".join(e.allowed_countries[:5])}

    if e.geo_scope == "worldwide" or "Worldwide" in e.allowed_regions:
        if e.timezone_utc_min is not None and not _tz_ok(e, country):
            return "not_eligible", "worldwide_tz", {"tz": _tz(e)}
        if not _role_level_worldwide(e, location):
            return "unclear", "company_remote", {}
        # "Anywhere" in the text, but the title or location lists regions: those regions win.
        if not ROLE_WORLDWIDE_LOCATION.search(location or ""):
            listed = sorted(
                {LISTED_REGIONS[m.lower()] for m in LISTED_REGION_RE.findall(f"{title} {location}")}
            )
```

```
            if listed:
                if regions.intersection(listed):
                    match = sorted(regions.intersection(listed))[0]
                    return "likely", "region", {"region": match}
                return "unclear", "conflict", {"places": ", ".join(listed)}
        return _cap(e, "eligible"), "worldwide", {}

    if e.allowed_regions:
        match = sorted(regions.intersection(e.allowed_regions))
        if match:
            return "likely", "region", {"region": match[0]}
        return "not_eligible", "regions_only", {"places": ", ".join(e.allowed_regions)}

    if e.timezone_utc_min is not None:
        if _tz_ok(e, country):
            return "likely", "tz_ok", {"tz": _tz(e)}
        return "not_eligible", "tz_no", {"tz": _tz(e)}

    if e.remote_type == "fully_remote":
        return "unclear", "remote_unclear", {}
    return "unclear", "unclear", {}
```

1. **Explicitly excluded** ("not available in X") beats everything.
2. **Office or hybrid**: likely if the office is in this country, otherwise not eligible.
3. **Work authorization required elsewhere** ("must be authorized to work in the US"): not eligible unless this country is in the list.
4. **The country is named**: eligible (capped to likely if confidence is low).
5. **A list of other countries**: not eligible.
6. **Worldwide**: check the time zones, check that "worldwide" describes this role and not the company, and that the title or location do not list regions; then eligible.
7. **Regions**: likely if one contains the country, otherwise not eligible.
8. **Time zone only**: likely if the window overlaps the country's offsets.
9. Otherwise **unclear**.

The helpers

Time zone overlap

pipeline/classify/verdict.py · _tz_ok

```
def _tz_ok(e: Extraction, country: dict) -> bool:
    """The window fits when it overlaps the country's offset range (or its one utc_offset)."""
    lo = e.timezone_utc_min if e.timezone_utc_min is not None else -12
    hi = e.timezone_utc_max if e.timezone_utc_max is not None else 14
    off = country.get("utc_offset")
    cmin, cmax = (off, off) if off is not None else country["tz"]
    return cmin <= hi and lo <= cmax
```

Two ranges [a, b] and [c, d] overlap when `c <= b and a <= d`. A posting asking for "CET plus or minus 2 hours" (UTC-1 to UTC+3) fits Tunisia (UTC+1) and Egypt (UTC+2), but not the US.

Company slogan or role rule?

"We are a work-from-anywhere company" says nothing about who this particular job hires. Before trusting a worldwide extraction, the code looks for role-level evidence in the location field or the quote:

pipeline/classify/verdict.py · _role_level_worldwide

```
def _role_level_worldwide(e: Extraction, location: str) -> bool:
    if ROLE_WORLDWIDE_LOCATION.search(location or ""):
        return True
    q = e.evidence_quote
    if ROLE_LEVEL_QUOTE.search(q):
        return True
    # Neither the location field nor a role-level sentence: a company slogan is not enough.
    return not COMPANY_LEVEL_QUOTE.search(q) and not location
```

pipeline/classify/verdict.py · _cap

```
def _cap(e: Extraction, v: Verdict) -> Verdict:
    """Low-confidence extractions never produce a firm "eligible"."""
    return "likely" if v == "eligible" and e.confidence == "low" else v
```

Worked examples (for Tunisia)

The posting says	Extraction (short)	Verdict
Location "Remote, EMEA"	regions: EMEA	likely ("Open across EMEA, which includes Tunisia")
"Remote (US). Must be authorized to work in the US"	countries US, auth US	not eligible
"Remote, worldwide. Overlap with UTC-1 to UTC+3"	worldwide, tz -1..3	eligible
"Hybrid, Tunis office 2 days a week"	hybrid, countries TN	likely ("Needs hybrid office days in Tunisia")
Location "Berlin", "we are a remote-first company"	countries DE	not eligible
"Remote" and nothing else	remote, scope unclear	unclear ("Ask the employer")

Reasons in three languages

`decide()` returns a reason *key* and parameters, never a sentence. `reason()` renders the sentence in English, French or Arabic. French templates never put an article or preposition before a country name ("{name_fr} figure parmi…"), so they are correct for "Tunisie", "Maroc" and "États-Unis" alike.

pipeline/classify/verdict.py · reason

```
def reason(key: str, params: dict, country: dict, lang: str) -> str:
    """Render a decided reason. Country names come from name, name_fr and name_ar."""
    fields = {k: country.get(k, country["name"]) for k in ("name", "name_fr", "name_ar")}
    fields |= params
    if lang == "fr" and "tz" in fields:
        fields["tz"] = fields["tz"].replace(" to ", " à ")
```

```
    if lang == "ar" and "tz" in fields:
        fields["tz"] = fields["tz"].replace(" to ", " إلى ")
    if "where" in params:
        fields["where"], fields["where_fr"] = WHERE[params["where"]]
        fields["where_ar"] = WHERE_AR[params["where"]]
    template = REASONS_AR[key] if lang == "ar" else REASONS[key][LANGS.index(lang)]
    return template.format(**fields)
```

Try it

Change one rule (for example, make regions give "eligible") and run `uv run pytest tests/test_classify.py -q`: the tests show exactly which real cases the change breaks. Because verdicts are computed at export time, a rule change needs no AI calls; the next export applies it to every job.

Chapter 13

Export: from the database to the website's data

`pipeline/export.py` turns about 25,000 open jobs into the files the site reads: a search index per language, 256 detail shards, and two metadata files. It never exports full descriptions.

Reading the jobs

pipeline/export.py · rows

```
def rows(conn: sqlite3.Connection):
    q = """SELECT j.id, j.title, j.company_name, j.location_raw, j.apply_url, j.first_seen_at,
                  j.source, e.data, j.board_slug, j.salary_min, j.salary_max, j.salary_currency,
                  j.department, j.employment_type_raw, j.remote_flag_raw, j.posted_at
           FROM jobs j JOIN extractions e ON e.content_hash = j.content_hash
           WHERE j.expired_at IS NULL"""
    for r in conn.execute(q):
        data = json.loads(r[7])
        verified = data.pop("evidence_verified", False)
        yield r, Extraction(**data), verified
```

Only jobs that are open and have an extraction are exported. The `evidence_verified` flag is taken out of the stored JSON before rebuilding the `Extraction`.

Verdicts for every country, compressed

Storing 243 verdicts per job for 12,000 jobs would be 3 million entries. Instead, countries with the same outcome are grouped:

pipeline/export.py · groups

```
def groups(e: Extraction, location: str = "", title: str = "") -> list[dict]:
    """Verdicts for every WORLD country, grouped by outcome. [] when no country may see it."""
    by: dict[tuple, list[str]] = {}
    for code, country in config.WORLD.items():
        v, key, params = decide(e, code, country, location, title)
        by.setdefault((v, key, tuple(sorted(params.items()))), []).append(code)
    if not any(k[0] in SHOWN for k in by):
        return []
    common = max(by, key=lambda k: len(by[k]))
    star = common[0] in SHOWN
    out = [
        {
            "v": v,
            "r": {lang: reason(key, dict(params), PLACEHOLDER, lang) for lang in LANGS},
            "c": "*" if star and k == common else codes,
        }
        for k, codes in by.items()
        for v, key, params in [k]
        if star or v in SHOWN
    ]
    return sorted(out, key=lambda g: RANK[g["v"]])
```

For a job open in EMEA, the result looks like this: one group "likely, Open across EMEA, which includes {c}" listing about 110 country codes, and a group `"*"` meaning "every other country". Reasons carry a `{c}` placeholder where the country name goes, filled in later in the visitor's language. A job no country may see (`[]`) is not exported at all.

pipeline/export.py · for_country

```
def for_country(gs: list[dict], code: str) -> dict | None:
    """The group that holds `code`, else the "*" group, else None (not eligible)."""
    star = None
    for g in gs:
        if g["c"] == "*":
            star = g
        elif code in g["c"]:
            return g
    return star
```

Building one job record

pipeline/export.py · build

```
def build(conn: sqlite3.Connection) -> list[dict]:
    """Every remote, hybrid or unclear job open to at least one country, newest first."""
    sites = websites()
    logos = {p.stem: p.name for p in LOGOS.glob("*")} if LOGOS.is_dir() else {}
    out = []
    for r, e, verified in rows(conn):
        jid, title, company, location, url, seen, source, _, board = r[:9]
        smin, smax, scur, dept, emp, _flag, posted = r[9:]
        if e.remote_type not in WORKPLACES:
            continue
        gs = groups(e, location or "", title)
        if not gs:
            continue
        key = slugify(company)
        out.append(
            {
                "id": jid,
                "title": title,
                "company": company,
                "location": location or "",
                "url": url,
                "seen_at": seen,
                "posted_at": posted,
                "slug": job_slug(jid, company, title),
                "company_key": key,
                "logo": logos.get(key, ""),
                "website": sites.get((source, board), ""),
                "salary": [smin, smax, scur] if smin or smax else None,
                "tz": [e.timezone_utc_min, e.timezone_utc_max, e.timezone_requirement]
                if e.timezone_utc_min is not None or e.timezone_requirement
                else None,
                "department": dept or "",
                "contract": emp or "",
                "workplace": e.remote_type,
                "named": e.allowed_countries,
                "source": source,
                "scope": scope_label(e),
                "family": e.role_family,
                "seniority": e.seniority,
                "engagement": e.engagement,
                "languages": e.languages_required,
                "confidence": e.confidence,
                # Only quote what really appears in the posting, and never a company
                # slogan: then the location field is the evidence.
                "quote": e.evidence_quote if verified and not _slogan(e.evidence_quote) else "",
                "groups": gs,
            }
        )
```

```
    out.sort(key=when, reverse=True)
    return out
```

- Office-only jobs are dropped (`WORKPLACES`).
- A short label like "EMEA" or "US · CA" (`scope_label`) summarises who may apply.
- The quote is shown only if it really appears in the posting and is not a company slogan.
- Jobs are sorted by the employer's posting date, newest first (`when`).

Slugs: readable, stable URLs

pipeline/export.py · job_slug

```
def job_slug(jid: str, company: str, title: str) -> str:
    return f"{slugify(f'{company} {title}')}-{hashlib.sha1(jid.encode()).hexdigest()[:6]}"
```

`/jobs/reddit-senior-growth-manager-contract-2a343b`: the company and title for humans and search engines, plus 6 hex characters of a SHA-1 of the job id so two jobs with the same title never collide. The first two of those characters choose the job's shard.

The files

pipeline/export.py · search_record

```
def search_record(j: dict, lang: str) -> dict:
    """Compact card record for one language's search index."""
    place = j["scope"] or ", ".join(p.strip() for p in j["location"].split(";")[:2])
    rec = {k: j[k] for k in ("id", "slug", "title", "company", "company_key", "family")}
    rec |= {k: j[k] for k in ("seniority", "engagement", "languages", "seen_at", "salary")}
    rec |= {"scope": place, "workplace": j["workplace"], "quote": _trim(j["quote"])}
    if j["posted_at"]:
        rec["posted_at"] = j["posted_at"][:10]
    if j["logo"]:
        rec["logo"] = j["logo"]
    rec["groups"] = [{"v": g["v"], "r": g["r"][lang], "c": g["c"]} for g in j["groups"]]
    return rec
```

pipeline/export.py · write_shards

```
def write_shards(jobs: list[dict], out: Path) -> None:
    by_company: dict[str, list[dict]] = defaultdict(list)
    by_family: dict[str, list[dict]] = defaultdict(list)
    for j in jobs:  # newest first
        by_company[j["company_key"]].append(j)
        by_family[j["family"]].append(j)
    shards: dict[str, dict] = defaultdict(dict)
    for j in jobs:
        co = by_company[j["company_key"]]
        similar = []
        for s in by_family[j["family"]]:
            if s["company_key"] != j["company_key"]:
                similar.append(_related(s))
                if len(similar) == RELATED:
                    break
        shards[shard(j["slug"])][j["slug"]] = j | {
            "company_n": len(co),
            "company_named": sorted({c for x in co for c in x["named"] if c in config.WORLD}),
            "more": [_related(x) for x in co if x["id"] != j["id"]][:RELATED],
            "similar": similar,
        }
```

```
    out.mkdir(parents=True, exist_ok=True)
    for old in out.glob("*.json"):
        old.unlink()
    for name, recs in shards.items():
        _dump(out / f"{name}.json", recs)
```

The search index holds only what a card needs, so it stays around 500 KB compressed per language. A job page needs more (location, apply link, time zone, department, similar jobs) and loads only its shard, one 256th of the details.

pipeline/export.py · export

```
def export(
    conn: sqlite3.Connection, out_dir: str | Path, public_dir: str | Path = "site/public/data"
) -> str:
    out_dir, public_dir = Path(out_dir), Path(public_dir)
    out_dir.mkdir(parents=True, exist_ok=True)
    public_dir.mkdir(parents=True, exist_ok=True)
    jobs = build(conn)
    for lang in LANGS:
        _dump(public_dir / f"search-{lang}.json", [search_record(j, lang) for j in jobs])
    write_shards(jobs, public_dir / "j")
    companies: dict[str, dict] = {}
    for j in jobs:
        c = companies.setdefault(
            j["company_key"], {"name": j["company"], "website": "", "n": 0, "entry": 0}
        )
        c["n"] += 1
        c["entry"] += j["seniority"] in ("intern", "junior")
        c["website"] = c["website"] or j["website"]
        c.setdefault("named", set()).update(x for x in j["named"] if x in config.WORLD)
    for c in companies.values():
        c["named"] = sorted(c["named"])
    _dump(out_dir / "companies.json", companies)  # build-time only: company pages
    boards = conn.execute(
        "SELECT ats, COUNT(*) FROM boards WHERE active = 1 GROUP BY ats ORDER BY 2 DESC"
    ).fetchall()
    open_jobs = conn.execute("SELECT COUNT(*) FROM jobs WHERE expired_at IS NULL").fetchone()[0]
    meta = {
        "updated": datetime.now(UTC).replace(microsecond=0).isoformat(),
        "product": config.PRODUCT_NAME,
        "domain": config.DOMAIN,
        "contact": config.CONTACT_EMAIL,
        "open_jobs": open_jobs,
        "jobs": len(jobs),
        "companies": len({j["company_key"] for j in jobs}),
        "boards": {a: n for a, n in boards},
        # Featured countries: Telegram channels and the getting-paid guide.
        "countries": {
            code: {
                "name": c["name"],
                "name_fr": c["name_fr"],
                "name_ar": c["name_ar"],
                "counts": dict(Counter(j["verdict"] for j in country_view(jobs, code))),
            }
            for code, c in config.COUNTRIES.items()
        },
        "world": {
            code: {k: c[k] for k in ("name", "name_fr", "name_ar", "regions")}
            for code, c in config.WORLD.items()
        },
    }
    (out_dir / "meta.json").write_text(json.dumps(meta, ensure_ascii=False, indent=1), "utf-8")
    lines = [f"## {config.PRODUCT_NAME} export", "", f"- {len(jobs)} jobs shown somewhere"]
    for code, m in meta["countries"].items():
```

```
        lines.append(f"- {code}: {m['counts']}")
    return "\n".join(lines)
```

Why not prerender every job page?

Cloudflare Pages' free plan allows 20,000 files per deployment. 12,000 jobs in 3 languages would be 36,000 pages. So job pages are built on request by a Function from the shards (chapter 17), and everything else is static.

Chapter 14

Email alerts, welcome email and Telegram

People can get new jobs delivered: a daily email digest per country (and optional field), a welcome email when they subscribe, and four Telegram channels. All three must never send the same job twice, never spam, and always offer a one-click way out.

Signing up (site/functions/api/subscribe.js)

site/functions/api/subscribe.js

```
import { COUNTRIES, FAMILIES, json, readForm } from "./_shared.js";
import { sendWelcome } from "./_welcome.js";

const EMAIL = /^[^\s@]{1,64}@[^\s@]{1,255}\.[^\s@]{2,}$/;

export async function onRequestPost({ request, env, waitUntil }) {
  const get = await readForm(request);
  if (!get) return json({ error: "bad request" }, 400);
  if (get("website")) return json({ ok: true }); // honeypot: quietly drop bots
  const email = get("email").toLowerCase();
  const country = get("country");
  const family = get("family");
  const lang = ["fr", "ar"].includes(get("lang")) ? get("lang") : "en";
  if (!EMAIL.test(email) || !COUNTRIES.includes(country) || !FAMILIES.includes(family)) {
    return json({ error: "invalid" }, 422);
  }
  const res = await env.DB.prepare(
    "INSERT OR IGNORE INTO subscribers (email, country, family, lang, created_at) VALUES (?, ?, ?, ?, ?)",
  )
    .bind(email, country, family, lang, new Date().toISOString())
    .run();
  // A new subscription (not a repeat of the same one) gets a welcome email, sent after the reply.
  if (res.meta?.changes) waitUntil(sendWelcome(env, { email, country, family, lang }));
  return json({ ok: true });
}
```

- A hidden **honeypot** field called "website": humans never fill it, bots usually do, so such sign-ups are dropped silently.
- `INSERT OR IGNORE` with the primary key (email, country, field) makes repeated sign-ups harmless.
- Only a truly new row (`res.meta.changes`) triggers the welcome email, sent with `waitUntil`: after the answer has gone back to the browser, so the form stays fast and a failed send can never break the sign-up.

The welcome email (site/functions/api/_welcome.js)

site/functions/api/_welcome.js · render

```
export function render({ email, country, family, lang }, unsubUrl) {
  const t = T[lang] ?? T.en;
  const c = meta.world[country] ?? { name: country, name_fr: country, name_ar: country };
  const name = { en: c.name, fr: c.name_fr, ar: c.name_ar }[lang] ?? c.name;
  const f = FIELDS[family]?.[LANGS.indexOf(lang)] ?? "";
  const v = { country: name, field: f ? fill(t.field, { field: f }) : "" };
  const page = `https://${meta.domain}${lang === "en" ? "" : "/" + lang}/jobs?c=${country}`;
  const html =
    `<div dir="${lang === "ar" ? "rtl" : "ltr"}" style="font-family:Arial,sans-serif;max-
width:600px;margin:auto;color:#14223a">` +
    `<p style="font-size:18px;font-weight:700;margin:0 0 12px">HiresHere</p>` +
```

```
    `<p style="font-size:15px">${esc(t.hello)}</p>` +
    `<p style="font-size:15px">${esc(fill(t.what, v))}</p>` +
    `<p><a href="${page}" style="display:inline-block;background:#0b3a53;color:#fff;padding:12px 20px;border-
radius:999px;text-decoration:none;font-weight:600">${esc(fill(t.now, v))}</a></p>` +
    `<p style="color:#5f6e84;font-size:12px">${esc(t.foot)}<br><a href="${unsubUrl}" style="color:#5f6e84">${t.unsub}
</a></p></div>`;
  const text = [t.hello, fill(t.what, v), `${fill(t.now, v)}: ${page}`, t.foot, `${t.unsub}:
${unsubUrl}`].join("\n\n");
  return { subject: fill(t.subject, v), html, text };
}
```

It explains what to expect (one email a day, only when new jobs open to their country appear), links to today's jobs, and includes the unsubscribe link. French sentences are worded so they are grammatical for every country name.

Unsubscribe links that cannot be forged

The link carries the email and a token: an **HMAC** (keyed hash) of the email made with a secret only the server knows. Anyone can see the link, but nobody can compute a valid token for someone else's address. The pipeline (Python) and the Function (JavaScript) produce identical tokens:

pipeline/alerts.py · unsubscribe_token

```
def unsubscribe_token(email: str, secret: str) -> str:
    """Must match site/functions/api/unsubscribe.js (HMAC-SHA256, hex)."""
    return hmac.new(secret.encode(), email.encode(), hashlib.sha256).hexdigest()
```

site/functions/api/unsubscribe.js · remove

```
async function remove(request, env) {
  const url = new URL(request.url);
  const email = url.searchParams.get("e") || "";
  const token = url.searchParams.get("t") || "";
  const key = await crypto.subtle.importKey(
    "raw", new TextEncoder().encode(env.UNSUBSCRIBE_SECRET), { name: "HMAC", hash: "SHA-256" }, false, ["sign"],
  );
  const sig = new Uint8Array(await crypto.subtle.sign("HMAC", key, new TextEncoder().encode(email)));
  const expected = [...sig].map((b) => b.toString(16).padStart(2, "0")).join("");
  if (!email || token !== expected) return false;
  await env.DB.prepare("DELETE FROM subscribers WHERE email = ?").bind(email).run();
  return true;
}
```

Emails also carry `List-Unsubscribe` and `List-Unsubscribe-Post` headers, so Gmail and others show their own one-click "Unsubscribe" button (RFC 8058).

The daily digest (pipeline/alerts.py)

Which jobs are "new" for a subscriber? Those that appeared on the site since their last digest. A job classified today from the backlog counts as new today, even if it was first fetched last week:

pipeline/alerts.py · visible_since

```
def visible_since(conn: sqlite3.Connection) -> dict[str, str]:
    """When each open job appeared on the site: first seen, or classified if that came later
    (a backlog job classified today is news today)."""
    return dict(
        conn.execute(
```

```
            "SELECT j.id, MAX(j.first_seen_at, COALESCE(e.created_at, '')) FROM jobs j "
            "LEFT JOIN extractions e ON e.content_hash = j.content_hash WHERE j.expired_at IS NULL"
        )
    )
```

pipeline/alerts.py · pick

```
def pick(jobs: list[dict], first_seen: dict[str, str], since: str, family: str) -> list[dict]:
    """New eligible/likely jobs for one subscriber, eligible first, newest first."""
    hits = [
        j
        for j in jobs
        if j["verdict"] in ("eligible", "likely")
        and first_seen.get(j["id"], "") > since
        and fresh(j)
        and (not family or j["family"] == family)
    ]
    hits.sort(key=lambda j: first_seen[j["id"]], reverse=True)
    hits.sort(key=lambda j: j["verdict"] != "eligible")
    return hits
```

pipeline/alerts.py · due

```
def due(last_sent_at: str | None, now: datetime) -> bool:
    """At most one digest per UTC day, however many runs. last_sent_at is the newest job in the
    last digest (always before the send), so an hours gap would skip a run GitHub started late."""
    # ponytail: a run just after midnight UTC can follow one late the day before; runs start 04:17.
    if not last_sent_at:
        return True
    last = datetime.fromisoformat(last_sent_at.replace("Z", "+00:00"))
    return last.astimezone(UTC).date() < now.date()
```

`due()` guarantees at most one digest per UTC day, however many times the pipeline runs. It compares calendar days, not hours: an earlier 20-hour rule skipped subscribers when GitHub started a scheduled run late after a manual one. `fresh()` (in export.py) keeps out jobs posted more than 14 days ago, so adding a new company does not flood people with its old postings. Resend's free plan allows 100 emails a day; the code stops at 95 and leaves the rest for the next run.

pipeline/alerts.py · render

```
def render(sub: dict, jobs: list[dict], unsub_url: str) -> tuple[str, str, str]:
    lang = sub["lang"] if sub["lang"] in T else "en"
    t, c = T[lang], config.WORLD[sub["country"]]
    name = {"en": c["name"], "fr": c["name_fr"], "ar": c["name_ar"]}[lang]
    field = FIELDS.get(sub["family"], ("", "", ""))[list(T).index(lang)]
    v = {"n": len(jobs), "country": name}
    v["field"] = t["field"].format(field=field) if field else ""
    subject = t["subject1" if len(jobs) == 1 else "subject"].format(**v)
    page = f"https://{config.DOMAIN}{'' if lang == 'en' else '/' + lang}/jobs?c={sub['country']}"
    shown = jobs[:MAX_JOBS]
    e = html.escape

    rows = "".join(
        f'<tr><td style="padding:14px 0;border-bottom:1px solid #dce2ea">'
        f'<a href="{e(j["url"])}" style="color:#14223a;font-weight:600;text-decoration:none">{e(j["title"])}</a>'
        f'<div style="color:#44546b;font-size:14px">{e(j["company"])}'
        f"{' · ' + e(j['scope']) if j['scope'] else ''}"
        f' · <strong style="color:{"#16794a" if j["verdict"] == "eligible" else "#9a5800"}">{t[j["verdict"]]}
</strong></div>'
        f'<div style="color:#44546b;font-size:14px;margin-top:4px">{e(j["reason"][lang])}</div></td></tr>'
        for j in shown
    )
    body = (
```

```
        f'<div dir="{"rtl" if lang == "ar" else "ltr"}" style="font-family:Arial,sans-serif;max-
width:600px;margin:auto;color:#14223a">'
        f'<p style="font-size:18px;font-weight:700;margin:0 0 12px">{config.PRODUCT_NAME}</p>'
        f'<p style="font-size:15px">{e(t["intro"].format(**v))}</p>'
        f'<table style="width:100%;border-collapse:collapse">{rows}</table>'
        f'<p><a href="{page}" style="color:#1d5bc7">{e(t["more"].format(**v))}</a></p>'
        f'<p style="color:#5f6e84;font-size:12px">{e(t["foot"])}<br>'
        f'<a href="{unsub_url}" style="color:#5f6e84">{t["unsub"]}</a></p></div>'
    )
    text = "\n\n".join(
        [t["intro"].format(**v)]
        + [
            f"{j['title']} - {j['company']} ({t[j['verdict']]})\n{j['reason'][lang]}\n{j['url']}"
            for j in shown
        ]
        + [f"{t['more'].format(**v)}: {page}", t["foot"], f"{t['unsub']}: {unsub_url}"]
    )
    return subject, body, text
```

Telegram (pipeline/telegram.py)

pipeline/telegram.py · message

```
def message(job: dict, lang: str) -> str:
    e = html.escape
    t = T[lang]
    prefix = "" if lang == "en" else f"/{lang}"
    lamp = "🟢" if job["verdict"] == "eligible" else "🟠"
    lines = [
        f"<b>{e(job['title'])}</b>",
        e(job["company"]) + (f" · {e(job['scope'])}" if job["scope"] else ""),
        f"{lamp} {t[job['verdict']]}: {e(job['reason'][lang])}",
        f"https://{config.DOMAIN}{prefix}/jobs/{job['slug']}",
    ]
    return "\n".join(lines)
```

Each featured country has a private channel; the bot is an admin of each. Posted jobs are recorded in the `telegram_posts` table. On a channel's very first run only the newest five jobs are posted and the rest marked as seen, so a new channel is not flooded; after that at most 15 per run.

Chapter 15

The website: Astro, pages and languages

The site is built with Astro into plain HTML, CSS and a little JavaScript. Most pages are generated once at build time; job pages are filled in by a Cloudflare Function on request. Everything exists in English, French and Arabic.

How Astro works

An `.astro` file has two parts: a **frontmatter** block between `---` lines, which runs at build time (on your computer or in CI, never in the visitor's browser), and an HTML template that can use its variables. Components are imported like functions. At build time Astro runs every page and writes static `.html` files to `dist/`. A `<script>` inside a component is bundled and runs in the browser.

site/astro.config.mjs

```
import { readFileSync } from "node:fs";
import { defineConfig } from "astro/config";

// The domain lives in pipeline/config.py and reaches the site through the exported meta.json.
const meta = JSON.parse(readFileSync(new URL("./src/data/meta.json", import.meta.url), "utf-8"));

// Static output; Cloudflare Pages serves /dist and runs /functions.
export default defineConfig({
  site: `https://${meta.domain}`,
  trailingSlash: "never",
  build: { format: "file" },
});
```

`build.format: "file"` writes `/jobs.html` instead of `/jobs/index.html`, and `trailingSlash: "never"` keeps URLs clean. The domain comes from the exported `meta.json`, so it is defined once, in the Python config.

Pages and routes

Each file under `src/pages/` is a URL. French and Arabic versions live under `fr/` and `ar/`, with translated slugs (`/fr/entreprises`, `/fr/a-propos`). A page is usually three lines: pick the language, wrap a shared component in the layout.

site/src/pages/jobs.astro

```
---
import JobSearch from "../components/JobSearch.astro";
import Layout from "../layouts/Layout.astro";
import { site } from "../data";
import { tr } from "../i18n";
const lang = "en";
---

<Layout lang={lang} page="search" title={`${tr(lang, "search_title")} | ${site.product}`} description={tr(lang,
"home_lede")}>
  <JobSearch lang={lang} />
</Layout>
```

Dynamic routes such as `companies/[key].astro` export `getStaticPaths()`, which returns one entry per company; Astro builds a page for each:

site/src/pages/companies/[key].astro

```
---
import CompanyPage from "../../components/CompanyPage.astro";
import Layout from "../../layouts/Layout.astro";
import { companies, site } from "../../data";
import { tr } from "../../i18n";

export function getStaticPaths() {
  return Object.keys(companies).map((key) => ({ params: { key } }));
}
const { key } = Astro.params as { key: string };
const lang = "en";
const co = companies[key];
const vars = { company: co.name, product: site.product, n: co.n };
---

<Layout
  lang={lang}
  page="company"
  country={key}
  title={tr(lang, "company_meta", vars)}
  description={tr(lang, "company_intro", vars)}
>
  <CompanyPage companyKey={key} lang={lang} />
</Layout>
```

Build-time data (site/src/data.ts)

site/src/data.ts

```
// Build-time access to the JSON written by `python -m pipeline.run export`.
// Job cards and job pages are rendered from the search index and detail shards in
// public/data; the build only uses the English index for lists, counts and the sitemap.
import index from "../public/data/search-en.json";
import companiesIndex from "./data/companies.json";
import meta from "./data/meta.json";
import { type Lang, LOCALE } from "./i18n";
import type { SJob } from "./scripts/cards";

export { SOURCES } from "./scripts/cards";

export const site = meta;
export const jobs = index as unknown as SJob[];

export interface Country {
  code: string;
  name: string;
  name_fr: string;
  name_ar: string;
}

/** Featured countries: getting-paid guide, Telegram channels. */
export const countries: Country[] = Object.entries(meta.countries).map(([code, c]) => ({
  code,
  name: c.name,
  name_fr: c.name_fr,
  name_ar: c.name_ar,
}));

/** Every country a visitor can pick, sorted by its name in `lang`. */
export function world(lang: Lang): { code: string; name: string }[] {
  const sort = new Intl.Collator(LOCALE[lang]).compare;
  return Object.entries(meta.world)
    .map(([code, c]) => ({ code, name: countryName(c, lang) }))
    .sort((a, b) => sort(a.name, b.name));
```

```
}

export function countryName(c: Omit<Country, "code">, lang: Lang): string {
  return { en: c.name, fr: c.name_fr, ar: c.name_ar }[lang];
}

/** Company pages: name, website, number of jobs, countries named in their postings. */
export const companies = companiesIndex as Record<string, { name: string; website: string; n: number; entry: number;
named: string[] }>;

export function companyJobs(key: string): SJob[] {
  return jobs.filter((j) => j.company_key === key);
}

export const companyKeys = () => [...new Set(jobs.map((j) => j.company_key))];

export function totalBoards(): number {
  return Object.values(meta.boards).reduce((a, b) => a + (b as number), 0);
}
```

Importing a JSON file in Astro reads it at build time. Lists, counts, company pages and the sitemap come from here; the browser loads the search index separately.

Three languages (src/i18n.ts, src/i18n-ar.ts)

Every piece of interface text is a key in a dictionary per language. `tr(lang, key, vars)` looks it up and fills `{placeholders}`; `href(lang, page)` builds the right URL for a page in a language:

site/src/i18n.ts · href

```
export function href(lang: Lang, page: Page, country?: string): string {
  // Countries and fields are filters on the one search page: ?c=TN&f=engineering.
  if (page === "country") return `${SLUGS.search[lang]}?c=${(country ?? "").toUpperCase()}`;
  if (page === "field") {
    const [c, f] = (country ?? "").split("/");
    return `${SLUGS.search[lang]}?c=${c.toUpperCase()}&f=${f}`;
  }
  if (page === "job" || page === "company") return SLUGS[page][lang] + (country ?? "");
  return SLUGS[page][lang];
}
```

site/src/i18n.ts · tr

```
export function tr(lang: Lang, key: Key, vars: Record<string, string | number> = {}): string {
  return strings[lang][key].replace(/\{(\w+)\}/g, (_, k) => String(vars[k] ?? `{${k}}`));
}
```

The French dictionary is typed `typeof en`, so TypeScript reports a missing translation. Arabic pages set `dir="rtl"` on `<html>`, and the CSS uses logical properties (`padding-inline-start`, `inset-inline-end`) so layouts mirror automatically.

The layout (src/layouts/Layout.astro)

Every page goes through one layout: the `<head>` with title, description, canonical URL and `hreflang` alternates (so Google knows the three language versions are the same page), Open Graph tags for link previews, the header with navigation, the "Your country" select and language switch, and the footer.

site/src/layouts/Layout.astro · lines 24 to 31

```
---

<!doctype html>
<html lang={lang} dir={lang === "ar" ? "rtl" : "ltr"}>
  <head>
    <!-- Lets CSS reserve room for script-rendered cards and maps before they arrive (no layout shift). -->
    <script is:inline>document.documentElement.classList.add("js")</script>
    <meta charset="utf-8" />
```

The tiny inline script adds a `js` class before anything is drawn. CSS uses it to reserve room for the cards and maps that scripts render later, which prevents layout shift (content jumping while loading).

Components

Component	What it renders
Home	Hero with search band, latest jobs for the visitor's country, the world map band, the story section, alert sign-up.
JobSearch, SearchBand	The search page: query box, filters (verdict, field, level, contract, languages, posted date, salary), result list, pagination.
CompanyPage, CompaniesIndex	One company's jobs, levels hiring, countries named; the list of all companies.
Alerts, ProfileForm, SavedJobs	Email alert form; a light profile kept in the browser; saved jobs and application tracker.
GettingPaid, Employers, Sources, HowItWorks, About, Contact	Content pages: how to get paid from Tunisia/Morocco, the employer form, data credits, the method, the story, contact details.
Logo, Avatar, Lamp	The pin-and-check mark; company logo or initials; the small verdict "lamp" icons.

The design system

`styles/global.css` defines design **tokens** as CSS custom properties (colours like `--brand: #0b3a53`, `--accent: #ff7a59`, the verdict colours, spacing and type steps) and base styles; `styles/components.css` styles cards, pills, filters, job pages. `DESIGN.md` documents the rules: verdict colours are only used for verdicts, the accent never is, every button is at least 44 px tall for touch.

Chapter 16

The browser code: search, filters, saved jobs

The search page filters about 12,000 jobs instantly, in the visitor's browser, without a server. This chapter walks through the TypeScript in `site/src/scripts/`.

Finding the visitor's country (country.ts)

site/src/scripts/country.ts · visitorCountry

```
export async function visitorCountry(valid: (code: string) => boolean): Promise<string> {
  const check = (c: string | null) => (c && valid(c) ? c : "");
  const url = new URLSearchParams(location.search).get("c");
  if (url !== null) return check(url.split(",")[0].toUpperCase());
  try {
    const saved = localStorage.getItem(KEY);
    if (saved !== null) return check(saved);
    const seen = sessionStorage.getItem("hh_geo");
    if (seen !== null) return check(seen);
  } catch {
    // Storage blocked: ask the server every time.
  }
  try {
    const { country } = await fetch("/api/geo").then((r) => r.json());
    sessionStorage.setItem("hh_geo", country ?? "");
    return check(country);
  } catch {
    return "";
  }
}
```

Priority order: `?c=` in the URL (shared links), then the visitor's last choice (localStorage), then the country seen earlier this session, and finally `/api/geo`, a one-line Function returning the country Cloudflare detects from the IP address. The choice is mirrored in a cookie (`remember()`) so server-rendered job pages can read it too.

Verdict for a country (cards.ts)

site/src/scripts/cards.ts · groupFor

```
export function groupFor<T extends { c: string[] | "*" }>(gs: T[], code: string): T | null {
  let star: T | null = null;
  for (const g of gs) {
    if (g.c === "*") star = g;
    else if (g.c.includes(code)) return g;
  }
  return star;
}
```

site/src/scripts/cards.ts · verdictsFor

```
export function verdictsFor(gs: { v: V; c: string[] | "*" }[], code: string): Verdict[] {
  if (!code) return SHOWN.filter((v) => gs.some((g) => g.v === v));
  const g = groupFor(gs, code);
  return g && g.v !== "not_eligible" ? [g.v] : [];
}
```

The same "find my group, else the star group" logic as in the Python export, now in TypeScript. With no country selected, a job shows if it is open anywhere.

Drawing a card

site/src/scripts/cards.ts · card

```
export function card(j: SJob, ctx: Ctx, shown: Verdict[] = SHOWN): string {
  const { lang, updated } = ctx;
  const titleId = `t-${j.id.replace(/[^a-zA-Z0-9]/g, "-")}`;
  const salary = salaryText(j.salary, lang);
  const jobHref = esc(href(lang, "job", j.slug) + (ctx.code ? `?c=${ctx.code}` : ""));
  return `<li class="card" data-id="${esc(j.id)}">${avatar(j.company, j.company_key, j.logo, 52)}<div class="card-
body">
<h3 class="card-title" id="${titleId}"><a href="${jobHref}">${esc(j.title)}</a></h3>
<p class="card-meta"><a href="${esc(href(lang, "company", j.company_key))}">${esc(j.company)}</a>${salary ? ` ·
${esc(salary)}` : ""}</p>
${tags(j, lang, updated)}</div>
<div class="card-side">${cardPill(j, ctx, shown)}${dateTag(j, updated, lang, "card-date")}
<button type="button" class="icon-btn save-btn" data-save="${esc(j.id)}" aria-pressed="false" aria-
label="${esc(`${tr(lang, "save")}: ${j.title}`)}" aria-describedby="${titleId}">${STAR}</button></div></li>`;
}
```

Cards are built as HTML strings. Every value from the data goes through `esc()`, which turns `< > & " '` into entities: job titles come from third parties, and inserting them raw would let a malicious posting inject script into the page (an **XSS** attack).

site/src/scripts/cards.ts · dateTag

```
export function dateTag(j: SJob, updated: string, lang: Lang, cls = ""): string {
  const old = daysSince(when(j), updated) > OLD_DAYS ? " is-old" : "";
  return `<time class="${cls}${old}" datetime="${esc(when(j).slice(0, 10))}">${esc(foundText(j, updated, lang))}
</time>`;
}
```

The search engine (search.ts)

`initSearch()` loads the index for the page's language, works out the visitor's country, and keeps the state (query, country, filters, sort, page) in the URL, so every view can be shared or bookmarked. The core is one function that decides whether a card passes the current filters:

site/src/scripts/search.ts · passes

```
function passes(k: Card, st: State, skip?: Group): boolean {
  const j = k.job;
  const words = st.q.split(/\s+/).filter(Boolean);
  return (
    k.vs.length > 0 &&
    (skip === "v" || k.vs.some((v) => st.v.includes(v))) &&
    (skip === "f" || !st.f.length || st.f.includes(j.family)) &&
    (skip === "s" || !st.s.length || st.s.includes(j.seniority)) &&
    (skip === "e" || !st.e.length || st.e.includes(j.engagement)) &&
    (skip === "l" || !st.l.length || st.l.every((l) => j.languages.includes(l))) &&
    (skip === "d" || !st.d || k.days < Number(st.d)) &&
    (skip === "sal" || !st.sal || !!j.salary) &&
    words.every((w) => k.q.includes(w)) &&
    (!useProfile || !profile || languagesFit(j.languages, profile.langs))
  );
}
```

The optional `skip` argument powers the **live counts** next to each filter: to show how many results "Data" would give, count the cards that pass every filter except the field filter. That is how a filter panel can say "Data (5)" before you click it.

Words in the query must all appear in the title or company (lower-cased). Sorting is "best match" (verdict rank, then newest) or "newest first" by the employer's date. Twenty results per page.

Saved jobs and the tracker (saved.ts, profile.ts)

site/src/scripts/saved.ts · initSaveButtons

```
export function initSaveButtons(root: ParentNode = document): void {
  const store = load();
  root.querySelectorAll<HTMLElement>("[data-save]").forEach((btn) => {
    const id = btn.dataset.save!;
    paint(btn, id in store);
    btn.addEventListener("click", () => {
      const s = load();
      if (id in s) delete s[id];
      else s[id] = { status: "saved", t: Date.now() };
      save(s);
      paint(btn, id in s);
    });
  });
}
```

Saved jobs and their status (saved, applied, interview, offer, rejected) live in **localStorage**, in the browser only: no account, nothing on the server. Every access is wrapped in `try`, because private browsing can block storage, and the site must keep working.

site/src/scripts/profile.ts · languagesFit

```
export function languagesFit(required: string[], spoken: string[]): boolean {
  return !spoken.length || required.every((l) => spoken.includes(l));
}
```

Maps (worldmap.ts)

site/src/scripts/worldmap.ts · colourMap

```
export function colourMap(el: HTMLElement, svg: string, paint: (code: string) => { fill: string; title?: string }):
void {
  if (!svg) return;
  if (!el.querySelector("svg")) el.innerHTML = svg;
  el.querySelectorAll<SVGPathElement>("path[id]").forEach((p) => {
    const { fill, title } = paint(p.id);
    p.setAttribute("fill", fill);
    const t = p.querySelector("title");
    if (t && title) t.textContent = title;
  });
}
```

`public/world.svg` has one `<path id="TN">` per country. The home page colours each country by how many jobs are open to it; a job page colours it by verdict, with the visitor in coral. `zoomTo()` changes the SVG `viewBox` to frame one country.

Chapter 17

Cloudflare: Pages, Functions, D1 and the edge

Cloudflare serves hireshere.app from data centres around the world. Static files cost nothing and have no limit; Functions (server code) are free up to 100,000 calls a day. Knowing which is which is the key to this chapter.

The project (site/wrangler.jsonc)

site/wrangler.jsonc

```
{
  // Cloudflare Pages project for the HiresHere site. Build output and Functions live in site/.
  "name": "hireshere",
  "pages_build_output_dir": "./dist",
  "compatibility_date": "2026-09-15",
  "d1_databases": [
    {
      "binding": "DB",
      "database_name": "hireshere",
      "database_id": "f399b8aa-2976-4d79-a1fd-7108b48caa3b"
    }
  ]
}
```

The Pages project serves `dist/` and binds the D1 database as `env.DB` inside Functions. Deploys use `npx wrangler pages deploy --branch main`.

Which URLs run code: _routes.json

site/public/_routes.json

```
{
  "version": 1,
  "include": ["/jobs/*", "/fr/jobs/*", "/ar/jobs/*", "/api/*"],
  "exclude": []
}
```

> **The day job pages broke**
>
> Without this file, the middleware ran on *every* request: each image, logo, stylesheet and data file. One page view cost 20 or more Function calls, crawlers walked the 37,000 job URLs, and the free 100,000 a day ran out. Cloudflare then "fails open": it stops running Functions and serves only static files, so every job page showed the 404 page. Limiting Functions to `/jobs/*` and `/api/*` made a page view cost one call.

Static redirects: _redirects

site/public/_redirects · lines 1 to 6

```
# Old country boards (before the worldwide search). Static redirects cost no Functions.
/tn /jobs?c=TN 301
/tn/ /jobs?c=TN 301
/tn/:field /jobs?c=TN&f=:field 301
```

```
/ma /jobs?c=MA 301
/ma/ /jobs?c=MA 301
```

site/public/_redirects · lines 38 to 43

```
# Job pages are rendered by Functions. If Functions are off (free daily limit), serve the shell,
# which renders the job in the browser. Functions take these paths first when they run.
/jobs/* /shell/en 200
/fr/jobs/* /shell/fr 200
/ar/jobs/* /shell/ar 200
```

Old addresses from before the worldwide search (`/tn`, `/fr/ma/engineering`) redirect with a 301 (permanent) to the search with the right filters. The last three lines are a **fallback**: if Functions are ever off, job URLs serve the static page shell instead of a 404 (status 200 means "rewrite": the URL stays, the content comes from the shell). When Functions run, they take these paths first. The www to hireshere.app redirect is a Cloudflare Redirect Rule, configured in the dashboard.

Middleware (functions/_middleware.js)

site/functions/_middleware.js

```
// One canonical address: www and the pages.dev preview host redirect to the apex domain.
// Old country boards (/tn, /fr/tn/engineering...) redirect to the search with the same filters.
import meta from "../src/data/meta.json";

const codes = Object.keys(meta.countries).map((c) => c.toLowerCase()).join("|");
const OLD = new RegExp(`^(/fr|/ar)?/(${codes})(?:/([a-z]+))?/?$`);

export async function onRequest({ request, next }) {
  const url = new URL(request.url);
  const canonical = meta.domain;
  if (url.hostname === `www.${canonical}` || url.hostname === "hireshere.pages.dev") {
    url.hostname = canonical;
    return Response.redirect(url.toString(), 301);
  }
  const old = url.pathname.match(OLD);
  if (old) {
    const [, prefix = "", code, field] = old;
    const to = new URL(`${prefix}/jobs`, url);
    to.searchParams.set("c", code.toUpperCase());
    if (field) to.searchParams.set("f", field);
    return Response.redirect(to.toString(), 301);
  }
  return next();
}
```

Job pages on request (functions/_job.js)

site/functions/_job.js · handler

```
export function handler(lang) {
  return async ({ request, env, params }) => {
    const url = new URL(request.url);
    const asset = (path) => env.ASSETS.fetch(new URL(path, url));
    const slug = String(params.slug ?? "");
    let job = null;
    if (SLUG.test(slug)) {
      const shard = await asset(`/data/j/${slug.slice(-6, -4)}.json`);
      if (shard.ok) job = (await shard.json())[slug] ?? null;
```

```
    }
    if (!job) {
      // Closed (or never existed): 410 Gone with open jobs like it, picked in the browser.
      const page = await asset(`/closed/${lang}`);
      return new Response(page.body, { status: 410, headers: { "content-type": "text/html; charset=utf-8" } });
    }
    const code = visitor(request, url);
    const { title, description } = jobMeta(job, lang, code, meta);
    const shell = await (await asset(`/shell/${lang}`)).text();
    // Function replacers: job text may contain "$", which string replacements would interpret.
    const html = shell
      .replace(/<meta name="robots" content="noindex"\s*\/?>/, "")
      .replaceAll("__SLUG__", () => slug)
      .replaceAll("__TITLE__", () => esc(title))
      .replaceAll("__DESC__", () => esc(description))
      .replace('<div id="job-slot"></div>', () => jobPage(job, lang, code, meta));
    return new Response(html, { headers: { "content-type": "text/html; charset=utf-8", vary: "Cookie" } });
  };
}
```

1. Validate the slug with a regular expression (never trust the URL).
2. Fetch its shard from the static files (`env.ASSETS.fetch`, which costs no Function call).
3. Unknown slug: the job closed. Answer **410 Gone** with the "This job has closed" page, which suggests similar open jobs.
4. Work out the visitor's country: `?c=`, then the cookie, then Cloudflare's guess (`request.cf.country`).
5. Take the static shell for the language and replace its placeholders with the job's title, description and the rendered body (`jobPage()`, shared with the browser). Replacement functions are used because job text may contain `$`, which string replacement would interpret.

site/functions/jobs/[slug].js

```
import { handler } from "../_job.js";

export const onRequestGet = handler("en");
```

The browser fallback

site/src/pages/shell/[lang].astro · lines 26 to 48

```
  // Normally functions/_job.js fills the slot on the server. When Pages Functions are off
  // (free daily limit used up), _redirects serves this shell as-is: render the job here.
  const slot = document.getElementById("job-slot");
  if (slot) {
    const lang = (document.documentElement.lang || "en") as Lang;
    const slug = location.pathname.split("/").filter(Boolean).pop() ?? "";
    const q = new URLSearchParams(location.search).get("c")?.toUpperCase();
    const saved = document.cookie.match(/(?:^|;\s*)hh_country=([A-Z]{2})/)?.[1];
    const code = [q, saved].find((c) => c && c in meta.world) ?? "";
    const shard = /^[a-z0-9-]{1,80}-[0-9a-f]{6}$/.test(slug)
      ? await fetch(`/data/j/${slug.slice(-6, -4)}.json`).then((r) => (r.ok ? r.json() : {})).catch(() => ({}))
      : {};
    const job = shard[slug];
    if (job) {
      const { title, description } = jobMeta(job, lang, code, meta as never);
      document.title = `${title} | ${meta.product}`;
      document.querySelector('meta[name="description"]')?.setAttribute("content", description);
      slot.outerHTML = jobPage(job, lang, code, meta as never);
    } else {
      location.replace(`/closed/${lang}?from=${encodeURIComponent(slug)}`);
    }
```

```
  }
  init();
```

If the shell arrives with its slot still empty (no Function ran), the browser fetches the shard and renders the same page itself. Visitors never notice; only search engines get a slightly weaker page until Functions return.

The API endpoints (functions/api/)

Endpoint	Does
`POST /api/subscribe`	Validates and stores an alert subscription in D1; sends the welcome email.
`GET/POST /api/unsubscribe`	Checks the HMAC token and deletes the subscriber (the POST form is the one-click button in mail apps).
`POST /api/report`	Stores a "wrong verdict / job closed" report from a job page.
`POST /api/employer`	Stores an employer's request to be listed, reviewed by hand.
`GET /api/geo`	Returns the visitor's country code as Cloudflare sees it.

site/functions/api/_shared.js

```
// Shared validation for the two form endpoints. Runs on Cloudflare Pages Functions.
// Countries come from the pipeline's export (every country in data/countries.json).
import meta from "../../src/data/meta.json";

export const COUNTRIES = Object.keys(meta.world);
export const FAMILIES = ["", "engineering", "data", "design", "product", "marketing", "sales", "support", "ops",
"other"];

export function json(body, status = 200) {
  return new Response(JSON.stringify(body), {
    status,
    headers: { "content-type": "application/json", "cache-control": "no-store" },
  });
}

export async function readForm(request) {
  const type = request.headers.get("content-type") || "";
  if (!type.includes("form")) return null;
  const form = await request.formData();
  return (key) => String(form.get(key) ?? "").trim();
}
```

Inputs are always validated on the server (allowed countries, fields, lengths), whatever the browser form already checked.

The D1 database (site/schema.sql)

site/schema.sql

```
-- Cloudflare D1 schema for signups and error reports (Phase 4: wrangler d1 execute).
CREATE TABLE IF NOT EXISTS subscribers (
  email TEXT NOT NULL,
  country TEXT NOT NULL,
  family TEXT NOT NULL DEFAULT '',
  lang TEXT NOT NULL DEFAULT 'en',
  created_at TEXT NOT NULL,
  last_sent_at TEXT,
```

```
  PRIMARY KEY (email, country, family)
);
CREATE TABLE IF NOT EXISTS reports (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  job_id TEXT NOT NULL,
  country TEXT NOT NULL,
  verdict TEXT,
  kind TEXT NOT NULL,
  note TEXT,
  created_at TEXT NOT NULL
);
CREATE TABLE IF NOT EXISTS employer_leads (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  company TEXT NOT NULL,
  careers_url TEXT NOT NULL,
  countries TEXT,
  email TEXT NOT NULL,
  note TEXT,
  created_at TEXT NOT NULL
);
```

Queries use **prepared statements** with `?` placeholders and `.bind()`, never string concatenation, which rules out SQL injection. From your computer: `npx wrangler d1 execute hireshere --remote --command "SELECT count(*) FROM subscribers"`.

Other Cloudflare pieces

- **Web Analytics**: visits, page views, referrers and Core Web Vitals (load speed, layout shift), without cookies.
- **Email Routing**: contact@hireshere.app forwards to the owner's inbox.
- **Pages secrets**: `UNSUBSCRIBE_SECRET` and `RESEND_API_KEY`, available to Functions as `env.X`.

Chapter 18

Automation: the daily GitHub workflow

One YAML file runs the whole system every day on GitHub's servers: download the database, fetch, publish, classify, publish again, notify, save the database, check the live site. Here it is in full, then step by step.

.github/workflows/daily.yml

```
name: daily
on:
  schedule:
    - cron: "17 4 * * *" # once a day, 04:17 UTC (GitHub often starts it hours late)
  workflow_dispatch:

concurrency:
  group: daily
  cancel-in-progress: false

permissions:
  contents: write # needed to create and update the data-latest release

env:
  GH_TOKEN: ${{ github.token }}
  TAG: data-latest
  JOBS_DB: jobs.db
  GEMINI_API_KEY: ${{ secrets.GEMINI_API_KEY }}
  CLOUDFLARE_API_TOKEN: ${{ secrets.CLOUDFLARE_API_TOKEN }}
  CLOUDFLARE_ACCOUNT_ID: ${{ secrets.CLOUDFLARE_ACCOUNT_ID }}
  RESEND_API_KEY: ${{ secrets.RESEND_API_KEY }}
  UNSUBSCRIBE_SECRET: ${{ secrets.UNSUBSCRIBE_SECRET }}
  TELEGRAM_BOT_TOKEN: ${{ secrets.TELEGRAM_BOT_TOKEN }}
  TELEGRAM_CHANNELS: ${{ secrets.TELEGRAM_CHANNELS }}
  # The snapshot holds full job descriptions, so it is stored encrypted on the release.
  DB_PASSPHRASE: ${{ secrets.DB_PASSPHRASE }}
  GROQ_API_KEY: ${{ secrets.GROQ_API_KEY }}
  MISTRAL_API_KEY: ${{ secrets.MISTRAL_API_KEY }}

jobs:
  fetch:
    runs-on: ubuntu-latest
    timeout-minutes: 110
    steps:
      - uses: actions/checkout@v7
      - uses: astral-sh/setup-uv@v10.2.0
        with:
          python-version: "3.12"
      - run: uv sync --locked --no-dev

      - name: Download previous database
        run: |
          if [ -n "$DB_PASSPHRASE" ] && gh release download "$TAG" --pattern jobs.db.gz.enc; then
            openssl enc -d -aes-256-cbc -pbkdf2 -iter 200000 -pass env:DB_PASSPHRASE -in jobs.db.gz.enc | gunzip >
jobs.db
          elif gh release download "$TAG" --pattern jobs.db.gz; then
            gunzip jobs.db.gz  # older unencrypted snapshot
          else
            echo "No previous database, starting fresh."
          fi

      - name: Fetch
        id: fetch
        run: uv run python -m pipeline.run fetch

      - uses: actions/setup-node@v7.1.0
```

```
        with:
          node-version: 24
          cache: npm
          cache-dependency-path: site/package-lock.json

      # Publish the fresh fetch right away (closed jobs gone, new data live) instead of after the
      # slow AI step; the site is published again with the newly classified jobs at the end.
      - name: Publish early, before the AI step
        if: env.CLOUDFLARE_API_TOKEN != ''
        continue-on-error: true
        run: |
          uv run python -m pipeline.run export
          cd site && npm ci --no-audit --no-fund && npx astro build
          npx wrangler pages deploy --branch main --commit-dirty=true

      # Classify new jobs on free tiers, one request at a time (no billing since 2026-10-11):
      # Gemini and Groq free tiers side by side, newest jobs first, at most 35 minutes.
      # Whatever is left waits for the next run. classify-collect picks up any old batch.
      - name: Classify
        if: env.GEMINI_API_KEY != ''
        env:
          GEMINI_RPM: "14"
          GROQ_RPM: "4"
          MISTRAL_RPM: "50"
          CLASSIFY_MINUTES: "35"
        run: |
          uv run python -m pipeline.run classify-collect
          uv run python -m pipeline.run classify --limit 500 --yes

      - name: Stats
        run: uv run python -m pipeline.run stats

      # Logos first: the export records which companies have one.
      - name: Logos for new companies
        continue-on-error: true
        run: uv run python -m pipeline.logos

      # Rebuild and publish the site from today's data.
      - name: Export
        run: |
          uv run python -m pipeline.run export
          # Fail loudly (GitHub emails the owner) rather than deploy an empty site.
          uv run python -c "import json; m = json.load(open('site/src/data/meta.json')); n = m['jobs']; assert n > 0,
'export has no jobs'; print(n, 'jobs exported')"

      - name: Build and deploy site
        if: env.CLOUDFLARE_API_TOKEN != ''
        working-directory: site
        run: |
          [ -d node_modules ] || npm ci --no-audit --no-fund
          npx astro build
          # Cloudflare sometimes fails to publish with an internal error: retry once.
          npx wrangler pages deploy --branch main --commit-dirty=true ||
            (sleep 30 && npx wrangler pages deploy --branch main --commit-dirty=true)

      - name: Email alerts
        if: env.RESEND_API_KEY != ''
        run: uv run python -m pipeline.run alerts

      - name: Telegram channels
        if: env.TELEGRAM_BOT_TOKEN != ''
        continue-on-error: true
        run: uv run python -m pipeline.run telegram

      # Keep the day's fetch and classification even when a later step (deploy, alerts) fails.
      - name: Upload database
        if: always() && steps.fetch.outcome == 'success'
        run: |
          gzip -9 -k jobs.db
```

```
          gh release view "$TAG" >/dev/null 2>&1 || gh release create "$TAG" \
            --title "Latest data" --latest=false \
            --notes "Rolling SQLite snapshot updated by the daily workflow. Not a code release."
          if [ -n "$DB_PASSPHRASE" ]; then
            openssl enc -aes-256-cbc -pbkdf2 -iter 200000 -salt -pass env:DB_PASSPHRASE -in jobs.db.gz -out
jobs.db.gz.enc
            gh release upload "$TAG" jobs.db.gz.enc --clobber
            gh release delete-asset "$TAG" jobs.db.gz --yes 2>/dev/null || true
          else
            gh release upload "$TAG" jobs.db.gz --clobber
          fi

      # Check the live site like a visitor; a failure fails the run, so GitHub emails the owner.
      - name: Check the live site
        if: env.CLOUDFLARE_API_TOKEN != ''
        run: uv run python -m pipeline.healthcheck
```

Step by step

Step	What happens	Why it is built this way
on: schedule, workflow_dispatch	Runs daily at 04:17 UTC, or by hand (`gh workflow run daily.yml`).	GitHub often starts scheduled runs hours late under load; the site copy says "every day", not "every morning".
concurrency: daily	Two runs never overlap.	Both would download, change and upload the same database.
env	Secrets become environment variables for every step.	Keys never appear in the code or the logs (GitHub masks them).
Download previous database	Fetch `jobs.db.gz.enc` from the release and decrypt it.	The database is the pipeline's memory; it must survive between runs.
Fetch	All 537 boards (about a minute).	Has an `id` so later steps can check it succeeded.
Publish early	Export, build and deploy right away.	Closed jobs disappear and fresh data is live about 3 minutes in, without waiting for the AI.
Classify	Free-tier AI, newest first, up to 35 minutes.	The slow part; paced under each provider's limits.
Stats, Logos	Report totals; fetch icons for new companies.	Logos are `continue-on-error`: a failure must not stop the run.
Export (with a check)	Write the JSON; fail if it has no jobs.	Better a red run and an email than an empty website.
Build and deploy	Astro build and `wrangler pages deploy`, retried once.	Cloudflare occasionally fails to publish with an internal error.
Email alerts, Telegram	Notify subscribers and channels.	Run after the deploy, so links point to live pages.
Upload database	Compress, encrypt, upload; delete any plain copy.	`if: always() && steps.fetch.outcome == 'success'`: a later failure never loses the day's work, and a broken download never overwrites the good snapshot.
Check the live site	`pipeline.healthcheck`	Fails the run (GitHub emails the owner) if a page is broken.

Encrypting the snapshot

The repository is public, so its releases are too, and the database holds full job descriptions that must not be republished. The snapshot is encrypted with AES-256 (OpenSSL, key derived with PBKDF2, 200,000 iterations) using the `DB_PASSPHRASE` secret. Locally:

Download and decrypt the database on your computer

```
gh release download data-latest -p jobs.db.gz.enc
openssl enc -d -aes-256-cbc -pbkdf2 -iter 200000 -pass env:DB_PASSPHRASE \
    -in jobs.db.gz.enc | gunzip > jobs.db
```

The health check (pipeline/healthcheck.py)

pipeline/healthcheck.py · problems

```
def problems(pages: dict[str, tuple[int, str]], jobs: list[str]) -> tuple[list[str], list[str]]:
    """(failures, warnings) from {path: (status, body)}. A job page served as the browser
    fallback shell still works for visitors, so it is a warning (Functions are off)."""
    fail, warn = [], []
    for path, (status, body) in pages.items():
        if status != 200:
            fail.append(f"{path}: HTTP {status}")
        elif path in jobs and SHELL in body:
            warn.append(f"{path}: served by the browser fallback (Pages Functions are off)")
        elif path in jobs and "Page not found" in body:
            fail.append(f"{path}: shows Page not found")
    return fail, warn
```

After each run it loads the home page, the search page, `/api/geo` and four job pages taken from the export just made, retrying for a minute while the deploy settles. A job page served by the browser fallback is a warning (it works for visitors); a non-200 answer or "Page not found" is a failure.

Chapter 19

Testing, linting and evaluation

The project's rule is "tests for anything that parses or decides, and no network in tests". 75 tests run in under two seconds. Separately, a hand-checked sample measures how often verdicts are right.

Running the tests

From the repository root

```
uv run pytest -q                                    # all tests
uv run pytest tests/test_classify.py -q -k verdict   # only tests whose name contains "verdict"
uv run ruff check . && uv run ruff format --check .  # style and mistakes
```

Fixtures: real feeds, saved once

`tests/fixtures/` holds a real response from each job source (Greenhouse for Doximity, Lever for Spotify, Ashby for Supabase…). Tests feed them to the connectors' `parse()`, so they test real formats without touching the network.

tests/conftest.py

```
import json
from pathlib import Path

import pytest

FIXTURES = Path(__file__).parent / "fixtures"


@pytest.fixture
def fixture():
    def load(name: str):
        return json.loads((FIXTURES / name).read_text(encoding="utf-8"))

    return load


@pytest.fixture(autouse=True)
def no_sleep(monkeypatch):
    async def instant(_):
        return None

    monkeypatch.setattr("pipeline.http.asyncio.sleep", instant)
```

A pytest **fixture** is a function whose return value is passed to any test that names it as a parameter. `autouse=True` applies `no_sleep` to every test: retry waits become instant, so tests about retries do not take seconds.

tests/test_connectors.py · test_greenhouse

```
def test_greenhouse(fixture):
    jobs = greenhouse.parse(fixture("greenhouse_doximity.json"), board("greenhouse", "doximity"))
    assert len(jobs) == 2
    j = jobs[0]
    assert j.id == "greenhouse:doximity:8132424"
    assert j.title == "Client Success Manager, Pharma"
    assert j.location_raw == "San Francisco, CA or Remote (U.S.)"
    assert j.department == "Sales & Client Success"
```

```
    assert j.apply_url.startswith("https://job-boards.greenhouse.io/doximity/")
    assert "<" not in j.description_text and "&lt;" not in j.description_text
    assert "About Doximity" in j.description_text
```

Testing the rules

Each of the five diff rules has its own test, against an in-memory SQLite database (`":memory:"`, created fresh for every test):

tests/test_db.py · test_rule3_failed_fetch_expires_nothing

```
def test_rule3_failed_fetch_expires_nothing(conn):
    db.apply_result(conn, ok(job(1), job(2)), "t1")
    d = db.apply_result(conn, fail(), "t2")
    assert (d.new, d.updated, d.expired) == (0, 0, 0)
    for i in (1, 2):
        r = row(conn, f"lever:acme:{i}")
        assert r["expired_at"] is None and r["last_seen_at"] == "t1"
    b = brd(conn)
    assert b["consecutive_failures"] == 1 and b["last_error"] == "HTTP 503"
    assert b["last_ok_at"] == "t1"
```

tests/test_db.py · test_idempotent_second_run

```
def test_idempotent_second_run(conn):
    jobs = [job(i) for i in range(5)]
    assert db.apply_result(conn, ok(*jobs), "t1").new == 5
    second = db.apply_result(conn, ok(*jobs), "t2")
    assert (second.new, second.updated, second.expired) == (0, 0, 0)
```

The verdict tests cover each branch of `decide()` with small hand-made extractions, including the traps found in real postings (company slogans, regions in the title, time zone edges). The free-tier tests use `monkeypatch` to replace the AI calls with fakes that run out of quota at a chosen moment, and check that the next provider takes over and the rest is left for later.

tests/test_classify.py · test_sync_falls_back_to_groq_then_leaves_the_rest

```
def test_sync_falls_back_to_groq_then_leaves_the_rest(monkeypatch):
    import asyncio
    import contextlib

    from pipeline import db
    from pipeline.classify import compat

    groq = compat.GROQ
    conn = db.connect(":memory:")
    jobs = [job(id=f"x:y:{i}", content_hash=f"h{i}") for i in range(5)]
    calls = {"gemini": 0, "groq": 0}

    async def gem(client, j):
        calls["gemini"] += 1
        if calls["gemini"] > 2:
            raise gemini.QuotaError("Gemini daily quota")
        return ex(), 10, 5

    async def grq(client, j):
        calls["groq"] += 1
        if calls["groq"] > 1:
            raise gemini.QuotaError("Groq daily quota")
        return ex(), 10, 5

    monkeypatch.setattr(gemini, "generate", gem)
```

```
    monkeypatch.setattr(groq, "generate", grq)
    monkeypatch.setattr(gemini, "make_client", lambda: contextlib.AsyncExitStack())
    monkeypatch.setattr(groq, "make_client", lambda: contextlib.AsyncExitStack())
    monkeypatch.setattr(compat, "EXTRA", [groq])
    monkeypatch.setattr(groq, "available", lambda: True)
    monkeypatch.setattr(config, "GEMINI_RPM", 0)
    t = asyncio.run(runner.run_sync(conn, jobs))
    assert t["done"] == 3 and t["left"] == 2 and t["errors"] == 0
    assert t["by"] == {config.CLASSIFIER_MODEL: 2, groq.model: 1}
    models = [r[0] for r in conn.execute("SELECT model FROM extractions ORDER BY content_hash")]
    assert models.count(groq.model) == 1
```

Measuring accuracy (pipeline/classify/evalset.py)

Tests prove the code does what it says; they do not prove the verdicts are right. For that, `evalset sample` draws a stratified sample of 200 classified jobs (some of each verdict) into `data/eval/eval.csv` with an empty `label` column. A person reads each original posting and writes the true verdict. Predictions go to a separate file, so labelling stays blind. `evalset score` then reports the precision of "eligible" and "not eligible".

> **Open item**
>
> The current 200 labels were drafted by an AI and still need a human check. Until then, accuracy figures should be read as provisional.

Chapter 20

Design decisions and trade-offs

Every system is a set of choices. Here are the important ones in HiresHere, what was chosen, and what it cost. The full dated log is `docs/DECISIONS.md`.

Decision	Chosen	Alternative	Why
Where jobs come from	Public ATS feeds and boards that allow reuse	Scraping LinkedIn or Indeed	Legal and stable; feeds are structured JSON; the same postings as the companies' own pages.
Database	SQLite file, carried between runs as a release asset	A hosted Postgres	Free, zero setup, fast for 25,000 rows; the pipeline runs once a day in one place.
AI's role	Extract facts once; rules decide per country	Ask the AI "is this open to Tunisia?" per country	243 countries at no extra cost; rules are testable and explainable; changing a rule needs no AI calls.
Caution	Low confidence capped at "likely"; quote must be verbatim	Trust the model	A wrong "You can apply" costs the user an evening and the site its credibility.
Website	Static Astro pages, data in JSON, search in the browser	A server-rendered app with a database	Free hosting, fast everywhere, nothing to keep running.
Job pages	Rendered on request by a Function from shards	Prerender every job	36,000 pages exceed the free 20,000-file limit.
Functions scope	Only `/jobs/*` and `/api/*`, with a browser fallback	Middleware on every request	The free 100,000 calls a day ran out once; now one call per page view, and the site survives if they run out again.
AI cost	Free tiers of three providers in parallel, paced	Paid Gemini batch ($1.85 per 4,500 postings)	No budget. Trade-off: about 300 to 500 postings a day, so big additions take a few days.
Queue order	Newest first	Oldest first	Oldest first left today's jobs waiting and alerts empty.
Publishing	Right after fetching, and again after the AI	Once at the end	Fresh data is live in 3 minutes instead of 40.
Accounts	None; saved jobs and profile in localStorage	User accounts	Less to build and secure; nothing personal stored on servers.
Email	One digest a day at most; welcome email; HMAC unsubscribe	Instant emails per job	Respectful of inboxes and of Resend's 100 a day.
Dependencies	stdlib + httpx + pydantic; Astro only	Frameworks for scraping, ORM, UI kits	Less to learn, update and secure.
Open source	Public repo, MIT licence; database encrypted	Private	Visibility for the project and its author; descriptions stay protected.

Known limits

- Free AI quotas cap how many new postings are read per day; a large batch of new companies takes several days to process.
- Verdicts are automated estimates; the site always says to confirm with the employer.
- The Arabic interface text and country names still need a native review.
- GitHub starts scheduled runs late on busy days.
- The sitemap (about 39,000 URLs) is approaching the 50,000-URL limit per file and will need splitting.

Chapter 21

Working on the project: common tasks

A practical cheat sheet. Run commands from the repository root unless noted.

First-time setup

Setup

```
uv sync                          # Python dependencies
cd site && npm install && cd ..   # website dependencies
cp .env.example .env             # then fill in the keys you need
# get the latest database (see chapter 18 for decryption)
gh release download data-latest -p jobs.db.gz.enc
```

Everyday commands

Goal	Command
Fetch a few boards to test	`uv run python -m pipeline.run fetch --limit 20`
See totals	`uv run python -m pipeline.run stats`
Classify a few postings	`uv run python -m pipeline.run classify --limit 10`
Write the website's data	`uv run python -m pipeline.run export`
Build the site	`cd site && npx astro build` (output in `site/dist`)
Deploy by hand	`cd site && npx wrangler pages deploy --branch main`
Run the daily job now	`gh workflow run daily.yml`
Preview an alert email	`uv run python -m pipeline.run alerts --dry-run` (writes alerts-preview.html)
Check the live site	`uv run python -m pipeline.healthcheck`
Query subscribers	`cd site && npx wrangler d1 execute hireshere --remote --command "SELECT count(*) FROM subscribers"`
Rebuild this guide	`python docs/guide/build.py`

How to...

Add a company

Add a row to `data/boards.csv` (ats, slug, name, website, source). The next run fetches it; `pipeline.logos` picks up its icon.

Add a job source (a new ATS)

1. Save a real response as `tests/fixtures/<ats>_<company>.json`.
2. Write `pipeline/connectors/<ats>.py` with `parse()` and `fetch()`, following an existing connector.

3. Register it in `connectors/__init__.py` and add a test in `tests/test_connectors.py`.
4. Check the source's terms allow reuse, and credit it on the Sources page if they ask.

Add a free AI provider

If it speaks the OpenAI chat format, add one `Provider(...)` to `compat.py` and to `EXTRA`, its key to GitHub secrets and the workflow's `env`, and a pace (`<NAME>_RPM`). Compare its answers with Gemini's on 20 postings before relying on it.

Change a verdict rule

Edit `decide()` in `verdict.py`, add a test for the new case, run the tests, then export: every job gets the new verdict without any AI call.

Add interface text

Add the key to `en` and `fr` in `i18n.ts` (TypeScript will complain if French is missing) and to `i18n-ar.ts`, then use `tr(lang, "key")`.

> **Before you push**
>
> - `uv run pytest -q` and `uv run ruff check .` pass.
> - The site builds (`npx astro build`).
> - No secret in the diff. If you changed the database locally, remember CI keeps its own copy on the release.
> - Log a significant decision in `docs/DECISIONS.md`.

Chapter 22

Check your understanding

Try to answer from memory, then check the chapter in brackets. If you can answer all of them, you understand how HiresHere works.

1. Why does a failed fetch never expire jobs, and which rule says so? (6)
2. What is a job's `content_hash` made of, and what two things does it allow the pipeline to skip? (6, 10)
3. Why is SmartRecruiters fetched page by page, and why stop at 10 pages? (7)
4. What does the pre-filter do, and why does it let intern and junior titles through even when they name only a city? (10)
5. The AI returns a quote that is not in the posting. What happens to the extraction, and how does that change the verdict? (10, 12)
6. Why are verdicts computed by rules at export time instead of asking the AI per country? (2, 12)
7. A posting says "Remote, worldwide" in its text but its location field is "Remote, AMER". What does Tunisia get, and which code decides it? (12)
8. Explain the `"*"` group in an exported job and why it exists. (13)
9. Why are job pages rendered by a Function instead of being prebuilt? (13, 17)
10. What happened when there was no `_routes.json`, and what are the two defences now? (17)
11. How does the unsubscribe link prove it belongs to the person clicking it? (14)
12. A subscriber signed up at 10:00; the pipeline is re-run by hand at 15:00 the same day. Do they get two digests? Why? (14)
13. What does the queue do when Gemini answers with its daily quota error halfway through a run? (11)
14. Why does the workflow publish the site twice in one run? (18)
15. Why is the database snapshot encrypted, and when is it uploaded even if a later step failed? (18)
16. Every value inserted into a card goes through `esc()`. What attack does that prevent? (16)
17. How does the search page show "Data (5)" next to a filter before you click it? (16)
18. Where would you look to find out why a decision was made? (4, 20)

Going further

- Run the pipeline locally on 20 boards and open `jobs.db` with any SQLite viewer.
- Pick one job on the live site, find its extraction in the database, and walk through `decide()` by hand for your country.
- Write a test for a verdict case you think is missing, and make it pass.